From Resemblance to Authority: Claim-Bounded Evidence for Android Software Lineage in AI-Mediated Supply Chains

HAOYI ZHANG, Xi'an Jiaotong-Liverpool University, China
HUAIJIN RAN, Nanyang Technological University, Singapore
XUNZHU TANG, University of Luxembourg, Luxembourg
KISUB KIM, Daegu Gyeongbuk Institute of Science and Technology (DGIST), Republic of Korea
TEGAWENDÉ F. BISSYANDÉ*, University of Luxembourg, Luxembourg

Android lineage questions are often phrased as one detection problem even though byte identity, component presence, resemblance, derivation, build provenance, release authorization, and behavioral assurance are different propositions. Existing surveys separately cover Android repackaging, third-party libraries, software-supply-chain security, and software bills of materials; this review therefore does not claim an uncovered detector literature. Instead, it asks which observation licenses which claim, under what transformation and trust assumptions, and which bindings are required when evidence is composed across layers. The retained map contains 97 scholarly records, including 76 primary, empirical, or system studies. Selected or scoped full-text readings of 87 works support source-specific extractions; 10 records retain legacy coding that is not content-verified by record structure or DOI identity. Coverage of 21 review or method records across seven dimensions remains incomplete, so the matrix does not establish a literature gap. The synthesis organizes these distinctions in a seven-level ladder spanning identity, composition, resemblance, derivation, process provenance, authorization, and behavioral assurance. It separates resemblance and composition from directional history; a common predecessor can remain compatible with similarity and chronology. Signing, transparency, and provenance can bind principals, steps, and artifacts, but do not establish semantic equivalence or benign behavior. AI-mediated development motivates counter-histories involving sibling generations, synthesized dependencies, hidden context and model drift; these require explicit accepted-output and source-to-APK bindings. We propose a cross-layer assurance pattern that binds transformation-aware comparison, component/version evidence, authenticated step records, release policy, and behavior-specific analysis to the same digest or documented predecessor relation. The contribution is a claim-centered organizing framework and evaluation blueprint, not a detector ranking or empirical validation of the combined architecture.

Additional Key Words and Phrases: Android, software lineage, application repackaging, software supply chain, provenance, third-party libraries, code generation, evidence validity

1 Introduction

Two Android packages may look alike for many legitimate and illegitimate reasons. They may be separate builds of one code base, applications that embed the same advertising library, products generated from one template, regional variants signed by different subsidiaries, siblings produced from the same prompt, or an original and an unauthorized derivative. A detector can be accurate about a shared feature while still being unable to distinguish among those histories. The central problem is therefore not merely whether two artifacts are similar. It is whether the observation is strong enough for the relationship asserted by an analyst, marketplace, developer, or incident responder.

*Corresponding author: Tegawendé F. Bissyandé.

Authors' Contact Information: Haoyi Zhang, Xi'an Jiaotong-Liverpool University, Suzhou, China, hyeliozhang@gmail.com; Huaijin Ran, Nanyang Technological University, Singapore, Singapore, huaijin003@e.ntu.edu.sg; Xunzhu Tang, University of Luxembourg, Luxembourg, Luxembourg, realdanieltang@gmail.com; Kisub Kim, Daegu Gyeongbuk Institute of Science and Technology (DGIST), Daegu, Republic of Korea, kisub.kim@dgist.ac.kr; Tegawendé F. Bissyandé, University of Luxembourg, Luxembourg, Luxembourg, tegawende.bissyande@uni.lu.

This distinction is established indirectly across several mature literatures. Repackaging research studies code reuse, graph similarity, resource resemblance, visual clones, watermarks, and malicious grafting [13, 21, 45, 95, 96]. Third-party-library (TPL) research studies how to separate application code from reused components and identify families and versions despite shrinking, partial inclusion, and obfuscation [6, 43, 46, 48, 87, 91]. Software-supply-chain research instead asks who performed a step, which materials were consumed, which artifact was produced, how repository metadata was authorized, and whether different observers receive consistent histories [35, 51, 65, 75]. These bodies of work are complementary, but their guarantees do not compose automatically.

AI-mediated development makes the boundary more visible. A text-to-application system can generate template-based MIT App Inventor projects from natural-language descriptions, while Android development benchmarks evaluate generated projects and coding agents [27, 59]. Code assistants may also introduce insecure code, propose nonexistent packages, reproduce sensitive material, or alter code through iterations whose causal history is not represented by the final APK alone [52, 56, 57, 66, 68]. An application can therefore be novel at the byte level, similar at the behavioral or user-interface level, composed partly of inherited code, and produced through an authorized but weakly recorded process. No single binary relation captures all of those facts.

The practical consequence is a recurring category error. A high similarity score is reported as proof of copying; a component match is treated as proof of application ancestry; a valid signature is treated as proof of rightful authorization; an SBOM entry is treated as proof that the listed code is present and reachable; or a reproducible build is treated as proof that the inputs were approved. Each inference may become defensible with additional evidence, but the missing premise is often implicit. Once hidden, it is difficult to benchmark, transfer to a new workflow, or challenge during an appeal.

This survey does not claim that Android repackaging or supply-chain security lacks prior synthesis. Li, Bissyandé, and Klein reviewed 57 repackaging approaches and introduced RePack to expose reproducibility and comparison weaknesses [42]. Wu et al. subsequently published a comprehensive survey of Android repackaging detection, including code- and resource-based techniques, datasets, metrics, and future directions [81]. Zhan et al. organized 74 Android TPL studies around tasks including module separation, family identification, and version attribution, while Zeng et al. broadened the view to TPL security in application software [86, 89]. Recent syntheses cover software-supply-chain attacks and defenses, secure-design properties, research directions, and cyber-supply-chain risk [29, 36, 38, 55, 79]; empirical SBOM studies additionally expose design and adoption problems in practice [7, 69]. A defensible contribution must therefore be narrower than "an updated repackaging survey" and different from a general supply-chain taxonomy.

The question retained here is claim-centered: *which observable evidence supports which Android lineage or assurance proposition, under what transformation and trust assumptions, and what bindings are required when evidence is composed across artifacts, components, build steps, release policy, behavior, and AI-mediated changes?* The 21-record prior-review matrix is incomplete: located passages establish selected coverage dimensions, while unread dimensions remain unresolved. It therefore cannot establish that the retained set lacks an integrated prior synthesis. The organizing question is motivated by distinct evidence objects and counter-histories, not by an absence claim.

The survey answers four research questions:

RQ1: Claim structure. What distinct lineage and assurance claims recur in Android analysis, and which evidence objects can support each claim?

RQ2: Transformation robustness. How do rewriting, obfuscation, resource changes, library reuse, compilation, and generation alter the validity of artifact-level evidence?

RQ3: Cross-layer composition. How can artifact analysis, component evidence, authenticated process records, release authorization, and behavior analysis be combined without overstating any layer?

RQ4: AI-mediated development. Which assumptions become fragile when source and mobile artifacts are generated or revised through nondeterministic, tool-using systems?

Existing frameworks already separate supply-chain artifacts, operations, actors, properties, and defenses, and explicitly discuss AI development [29, 55, 79]. Building on those distinctions, this review offers an Android-focused seven-level claim ladder; a mechanism map linking objects, transformations, oracles, ceilings, and counter-histories; a digest-bound cross-layer assurance pattern; and an analysis of AI-mediated sibling generation, synthesized dependencies, hidden context, tool state, and model drift. The ladder adds propositions rather than ranking every higher level as universally better.

The map retains 97 records, including 76 primary/system studies and 21 review/method sources. Selected or scoped full-text readings have been checked for 87 works; the remaining 10 legacy extractions are not thereby substantively verified. The inventory is neither a complete search denominator nor independent dual coding.

The synthesis is that lineage evidence is *non-substitutable*: artifact and component evidence recover relationships when process records are absent; authenticated records establish direction and accountability; release controls bind distribution authority; and behavior analysis addresses security properties. Composition strengthens assurance only when artifacts, principals, times, policies, and transformations are explicitly bound.

Section 2 gives the review design. Sections 3–7 develop the claim model and domain syntheses; Sections 8 and 9 derive the cross-layer blueprint and agenda.

2 Review Design and Corpus

2.1 Review type, objective, and unit of analysis

This work is a bounded, claim-centered structured evidence synthesis, not a PRISMA-complete review or prevalence meta-analysis. Its analytic unit is a scholarly work contributing an Android artifact-comparison, component/version, provenance or reproducibility mechanism; an empirical supply-chain or AI-development study; or a structuring review or method source.

The coded unit relates object, observation, transformation model, oracle or policy, claim ceiling, and counter-history. Tool-level coding would hide stages such as module separation, signature construction, and version comparison; feature-level coding alone would also hide that clone pairs, family labels, repositories, signed policies, and rebuilds establish different propositions.

2.2 Search, snowballing, and corpus construction

Seeds included the Android repackaging benchmark review [42], the Android TPL review [89], broader Android surveys [70, 72, 83], and static-analysis/testing reviews [34, 44]. Backward and targeted forward snowballing followed supply-chain attacks, secure design, authenticated updates, provenance, transparency, reproducible builds, CI, package ecosystems, SBOMs, and AI-mediated development, guided by established review methods [9, 32, 80].

A 2024–2026 adversarial update added direct or adjacent syntheses [23, 38, 61, 79, 81, 86] and empirical SBOM studies [7, 69]. A candidate SBOM review was excluded after its official record reported withdrawal and no paper was available for coding. The artifact preserves 105 screening decisions plus source-location and retrospective discovery rows for every retained work; these support record audit, not a database-hit denominator.

Table 1. Corpus strata and their role. Counts describe the retained evidence map, not field prevalence.

Stratum	Works	Main evidence objects	Principal use
Android artifact analysis	34	DEX code, graphs, resources, UI, behavior, signatures, clone pairs	Bound artifact-level identity, resemblance, and directional claims
Third-party libraries	17	Module boundaries, signatures, versions, update histories	Separate composition, version, reachability, and application ancestry
Supply-chain integrity	17	Metadata, attestations, logs, roles, builds, CI workflows	Establish principals, steps, authorization, transparency, and reproducibility
AI-mediated development	8	Prompts, generated source, dependencies, interactions, accepted output	Identify generated transformations, missing context, and causal-record limits
Reviews and method	21	Taxonomies, protocols, benchmark critiques, design properties	Adversarially position scope and normalize evaluation obligations

The map contains 97 works: 76 primary/system and 21 review/method records. 74 primary/system and 93 total records are venue-published; the remainder are three preprints and one method report. Legacy depth labels record 61 full-or-selected-source and 36 abstract-or-selected-passage records. These broad labels do not certify which passages were read. The current content-check ledger identifies 87 works with actual PDF page/section or numbered primary-HTML section evidence and 10 legacy records not substantively reverified.

The heterogeneous corpus is not pooled into a detector ranking: candidate generation, transformations, labels, periods, and counting units differ, and benchmark construction can dominate apparent results [8, 28, 42]. The synthesis therefore compares mechanisms and inference validity.

2.3 Corpus records

The evidence package separates the representation consumed for coding from the canonical bibliographic record. Each retained work has a corpus row, BibTeX entry, source-location boundary, retrospective discovery row, and publication-status record. DOI-bearing entries use the DOI as canonical identifier; other records use an official venue, archive, or report locator. Automated checks require title and year agreement, unique identifiers, and consistency between venue/preprint status and entry type. A withdrawn SBOM candidate is preserved only in an exclusion ledger and historical regression fixture. These checks establish record-level consistency, not a comprehensive retraction-registry search or independent confirmation of every interpretation.

2.4 Prior-review adversary matrix

The 21 review/method records are compared across Android repackaging, component/version evidence, provenance, authorization, behavior, AI mediation, and an explicit claim ceiling with cross-layer binding. Table 2 uses 1 for located direct treatment, P for located partial treatment, 0 for absence supported by a complete scoped reading, and ? for unresolved. Located readings support 49 direct/partial cells and two scoped absences; 96 cells remain unresolved. Seventeen records have selected or scoped content checks, including four method sources whose seven coverage dimensions remain unresolved; four records have no current substantive check.

No all-dimension witness is established by the checked cells, but unresolved rows could supply one. Direct coverage is therefore undetermined, not a demonstrated gap. The matrix does not support a retained-set or worldwide absence claim.

Table 2. Selected prior-review adversary matrix. A=Android repackaging, C=component/version, P=process provenance, U=authorization, B=behavior, G=AI-mediated generation, X=claim-ceiling and binding rule.

Prior synthesis	A	C	P	U	B	G	X
[42]	1	P	?	?	P	?	?
[89]	P	1	?	?	P	?	?
[29]	?	P	1	1	1	?	1
[55]	0	P	1	1	P	0	P
[81]	1	1	?	?	P	?	P
[86]	?	1	?	?	P	?	?
[61]	?	?	?	?	?	?	?
[79]	?	1	1	1	P	1	P
[38]	?	P	P	P	P	?	?
[23]	?	?	?	?	?	?	?

2.5 Inclusion, exclusion, and source-depth rules

Works were included when their technical object could support an Android lineage or release claim. Clone/malware systems supplied reusable artifact observations; supply-chain systems supplied source, build, repository, or release evidence; and AI papers generated mobile software or measured security-relevant generation workflows. General model, GUI-agent, or prompt-injection work without a software-artifact lineage object was excluded.

Publication status and evidential role were separate. Preprints may expose emerging workflows, and method reports may guide procedure, without counting as peer-reviewed primary evidence. Every record has a scholarly/official locator and version-status row; withdrawn or retracted candidates are quarantined. This is not a comprehensive retraction-registry search, and lighter-depth records are limited to what their accessible materials support. Retained but substantively unchecked records [19, 23, 50, 54, 61, 62, 70, 73, 82, 84] contribute to the bibliographic inventory, not to technical conclusions or absence coding.

2.6 Claim-level extraction and recheck

Each retained record has seven fields: the evidence object, observation, transformation model, oracle or policy, claim ceiling, counter-history, and inference limit. Populating these fields is not equivalent to reading a primary source. The 87 checked works name the actual inspected PDF pages and sections or numbered primary-HTML sections; other extraction rows remain legacy assignments. Source-specific statements are distinguished from broad stratum categories: a category such as “renaming or library reuse” does not establish that the cited study tested every listed transformation. Counter-histories and inference limits are our analytical classifications, not empirical findings attributed to the source. Similarity alone establishes resemblance; direction requires chronology, source, ownership, or provenance. Authenticated builds do not prove safety, and family, version, and reachability remain distinct.

Three source-specific examples illustrate the distinction. FlowDroid concerns context-, flow-, field-, and object-sensitive taint analysis of Android callbacks and lifecycle behavior [5]; it is not a renaming experiment merely because it shares the Android-artifact stratum. LibScout builds package/class profiles from pruned method descriptors; its hierarchy-based matching can be defeated by package flattening and cannot distinguish every code-only version change [6]. ATVHunter combines library identification with version-sensitive analysis, so library presence alone does not establish its version-level conclusion [87]. Their extraction rows name the source sections and relevant conditions. The remaining legacy broad-category fields organize candidate mechanisms but do not enumerate transformations tested by each source; they are not used to infer per-operator robustness.

Table 3. Evidence-strength notation. A higher grade is not automatically available for every question.

Grade	Evaluation basis	Typical strength	Typical limitation
E0	Illustrative example	Makes a mechanism concrete	No discriminating evaluation
E1	Curated positive cases	Demonstrates feasibility	Does not estimate false positives or open-world behavior
E2	Positive and negative labeled corpus	Supports discrimination under a stated oracle	Labels may encode the method's assumptions
E3	Transformation-aware or adversarial corpus	Tests robustness to explicit changes	Covers a finite transformation family
E4	Temporal, cross-market, or independently constructed validation	Reduces leakage and construction bias	Transfer beyond sampled ecosystems remains uncertain
E5	Authenticated process evidence or independent reproduction	Supports a stated provenance or equality proposition	Does not imply semantic safety or policy authorization

The cumulative ledger contains 90 dated recheck entries (92.8%). This historical count is distinct from the 87 works checked against selected or scoped full-text readings in the current content ledger. Rechecks record the old and revised ceiling, source, result, and boundary; they are same-process checks, not independent coding.

2.7 Evidence-strength notation

The claim ladder states *which proposition* is supported; an orthogonal grade states *how it was evaluated*.

The two axes must remain separate. A valid signature may strongly bind a key and artifact while saying nothing about functional equivalence. A transformation-aware benchmark can strongly evaluate resemblance robustness without identifying the authorized publisher.

2.8 Synthesis and contradiction procedure

Synthesis first grouped observation families, then assigned each a ceiling and counter-history, and finally mapped the bindings needed across the Android lifecycle. Contradictions denote incompatible objects, denominators, times, policies, or oracles rather than numerical disagreement alone. This is not vote counting: repeated observations cannot convert resemblance into direction or provenance into benign behavior. Seven synthesis propositions retain named support, counter-histories, and boundaries in the claim ledger.

2.9 Deterministic traceability and negative controls

The standalone artifact checks one-to-one bindings among corpus, bibliography, extraction, source-location, provenance, status, screening, recheck, calibration, and material-claim ledgers. It rejects duplicate keys or canonical identifiers, title/year drift, missing source boundaries, publication-type conflicts, stale leads, generic extraction templates, or leakage of an excluded publication into active synthesis data. Mutation tests deliberately introduce representative faults to confirm that the checks fail rather than merely report success on the supplied files.

A retained historical gap pilot also exercises unknown handling. Its exhaustive 3×3 ternary-matrix oracle compares the implemented direct/mosaic classification with an independently expressed completion rule over every small case. This finite check validates bookkeeping semantics only; it does not prove literature completeness, source interpretation,

or any detector guarantee. Keeping manuscript totals current requires rerunning the checks and regenerating tables and count macros on the current evidence map before building. The ordinary paper build imports existing fragments; it does not itself enforce their freshness against current inputs.

The current offline suite passes 69 unit and mutation tests without skips; this does not validate source interpretation. Corpus checks retain all 97 records. The retained historical finite-oracle run covers 19,683 ternary matrices and 262,144 binary completions without disagreement; these totals are not a new empirical experiment. These are record-consistency and bookkeeping checks, not an additional source-reading or independent coding pass.

2.10 Validity limits

The map is not an exhaustive detector census: it lacks a complete search denominator, independent second coder, pooled accuracy estimate, and prevalence claim; 10 legacy records lack a current substantive content check, including four review/method records without a current substantive check; unread dimensions within checked records also remain unresolved. The matrix cannot establish a literature gap, and the source checks do not constitute independent interpretation.

3 An Evidence Model for Mobile Software Lineage

3.1 Why lineage is not one relation

The term *lineage* is used for at least four different relations. A structural relation states that two artifacts share material. A historical relation states that one artifact was derived from another or that both descend from a common source. A procedural relation states that an artifact was produced by a recorded sequence of steps and principals. A normative relation states that the production or release was authorized. Security and policy questions add a fifth relation concerning behavior. These relations can coincide, but none entails all the others.

Let an evidence item be represented as

$$e = (o, \phi, r, m, q, a),$$

where o is the observed object, ϕ is the feature or record extracted from it, r is the relation being inferred, m is the transformation model, q is the oracle or policy against which the inference is evaluated, and a is the set of assumptions. The tuple is not a new detector. It is a discipline for stating what a detector or provenance mechanism actually observes.

For example, a fuzzy code signature may compare methods after identifier normalization. Here, o is DEX method code, ϕ is a normalized signature, m includes identifier renaming but may exclude control-flow restructuring, and q may be a corpus of known repackaged pairs. The supported relation is resemblance under m. Direction is not part of that observation unless the oracle or auxiliary evidence supplies it. By contrast, an in-toto link observes authenticated step metadata and binds materials to products under a layout [75]. It supports a process relation under key, layout, and metadata assumptions, but does not establish that two programs are semantically equivalent.

3.2 The seven-level claim ladder

Figure 1 summarizes seven numbered claim categories. They are neither a maturity model nor an implication chain: a behavior claim can be supported without reconstructing provenance, and authorization does not establish resemblance. The order introduces different propositions, not cumulative guarantees.

L0*: Identity.* Cryptographic hashes, package digests, and reproducible-build comparisons can establish equality of the observed bytes with negligible collision probability under the selected hash assumptions. Identity is useful for artifact

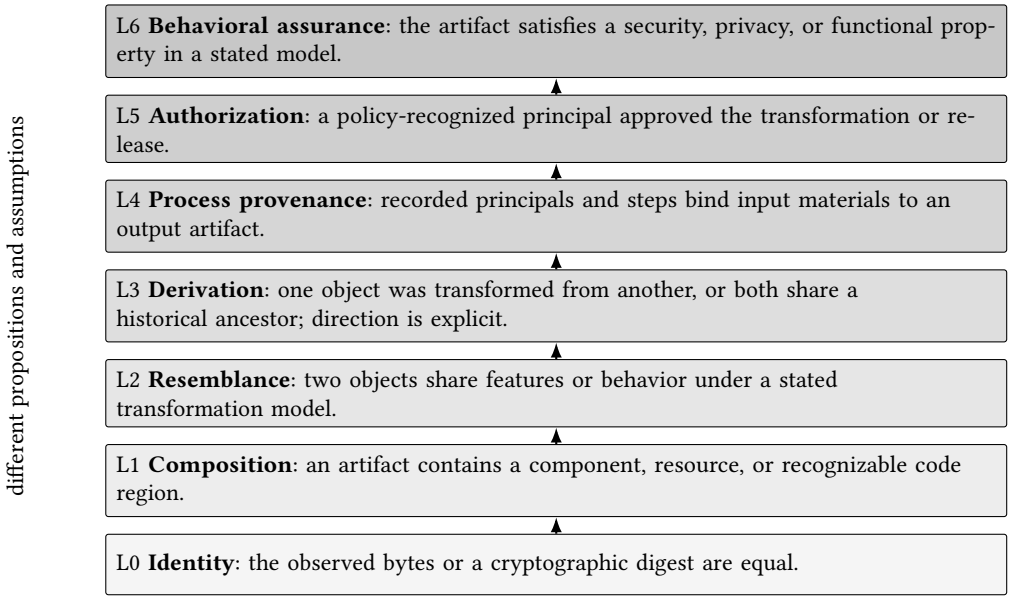


Fig. 1. Claim categories for Android lineage and assurance. Arrows order the topics, not logical implications. Each category needs its own evidence and assumptions; a higher number or similarity score does not establish the other propositions.

distribution and cache verification. It does not reveal why the bytes are equal, who authorized them, or whether they are safe. Two parties can distribute the same malicious binary.

L1*: Composition.* Component detectors, software bills of materials, package manifests, and static analyses address whether an artifact contains a component or a recognizable region. Android library research shows that composition is itself difficult because package structure can be flattened, names can be changed, dead code can be removed, and a library can be partially embedded [6, 77, 89, 91]. Composition also has granularity: identifying a library family is weaker than identifying an exact version, and identifying bytes is weaker than showing that vulnerable code is reachable.

L2*: Resemblance.* Most clone and repackaging detectors occupy this level. They establish that code, graphs, resources, UI, or behavior are similar under a threshold and transformation model. DroidMOSS, Juxtapp, AnDarwin, WuKong, SimiDroid, and related systems illustrate different feature choices [14, 26, 41, 76, 96]. Resemblance can be highly probative, particularly when independent feature families agree, but it does not by itself provide direction.

L3*: Derivation.* A derivation claim states that B descends from A, that A descends from B, or that both descend from a specified ancestor. Authenticated input–output records or controlled benchmark construction can establish direction under their trust assumptions. Timestamps, market chronology, ownership, and deployed marks can support adjudication, but order or ownership alone does not exclude a shared predecessor. Repackaging papers often use known pairs or market metadata to operationalize derivation; the feature detector and pair-construction oracle should be reported separately [28, 42].

L4*: Process provenance.* Process provenance binds materials, commands, environments, and principals to products. in-toto links and layouts are a direct example [75]; CI workflow studies show why the integrity of the recording and

execution environment also matters [33]. Provenance can make direction explicit because it records a step from inputs to output. It remains conditional on key control, policy, collection completeness, and the absence of unrecorded paths.

L5*: Authorization.* Authorization adds a policy judgment: the principal was allowed to perform or approve the step or release. TUF's delegated roles and Diplomat's repository delegations exemplify policy-scoped authority [35, 65]. An Android APK signature establishes possession of a signing key for that package artifact; whether the key holder is the rightful publisher is an external identity and policy question. Authorization is therefore not reducible to the presence of any valid signature.

L6*: Behavioral assurance.* Security, privacy, and functionality concern what the artifact does. Static taint analysis, malware classification, policy analysis, dynamic testing, and formal verification can support such claims under their models [4, 5, 49]. A provenance record may help select the artifact and inputs to analyze, but it cannot replace the behavior analysis. Conversely, a behavior detector may find malicious functionality without reconstructing the artifact's full history.

3.3 A worked claim-adjudication example

Consider two Android packages, A and B, collected from different channels. Their digests differ; both contain the same advertising-library family; normalized first-party call graphs, layouts, and images correspond after localization. Package A also has an attested build from a named source revision and a store release under a delegated publisher role. Package B uses another key and lacks an authenticated production record. Dynamic analysis reports that B contacts an additional endpoint. This synthetic example illustrates adjudication rather than reporting a measurement.

The observations license separate statements. Different digests reject L0 identity. The shared library supports family-level L1 composition while leaving version, modification, and reachability unresolved. Agreement across first-party code and resources supports L2 resemblance under the detector's transformation model, but does not select a direction: a common template, common predecessor, authorized fork, and copying either way remain compatible histories. The authenticated record supports L4 only for the materials, steps, principals, and product digest it names; delegation can add scoped L5 authorization for A. Neither claim transfers to B, and a different valid key is not by itself proof of unauthorized release. The endpoint finding is a L6 observation about the analyzed build and environment, not a derivation oracle.

Later evidence should revise only the subclaim it resolves. An authenticated parent and ordered transformation could establish directional L3 or process-level L4; a shared authenticated predecessor would instead establish sibling derivation. The correct report therefore preserves competing histories, binds every result to its exact object, and keeps retrieval, historical adjudication, authorization, and behavior distinct even when a store policy combines them into one action.

3.4 Claim ceilings of common evidence objects

Table 4 records typical claim ceilings. "Typical" matters: an implementation can combine additional evidence and move beyond the base ceiling. The purpose is to prevent that additional evidence from disappearing inside the name of the base mechanism.

Table 4. Typical claim ceilings and missing links. The last column names the evidence normally required for a stronger inference.

Evidence object	Ceiling	What it directly supports	Evidence needed for a stronger claim
Exact digest or byte comparison	L0	Identity of observed bytes	Authenticated origin, policy, or behavior analysis
Component signature or SBOM entry	L1	Component presence or asserted composition	Exact version, reachability, and authenticated generation
Code, graph, resource, UI, or trace similarity	L2	Resemblance under a feature and transformation model	Directional oracle, chronology, ownership, or process record
Known original–derivative pair	L3	Derivation for the constructed pair	Generalization to open-world candidates and authorization
Authenticated step attestation	L4	Recorded input–step–output relation and principal	Authorization policy, complete capture, and trusted execution
Delegated signature or release approval	L5	Policy-scoped authorization of metadata or artifact	Semantic equivalence, vulnerability, privacy, and behavior evidence
Static, dynamic, or formal behavior evidence	L6	Property within the analysis or test model	Broader environment coverage and provenance of analyzed inputs

3.5 Counter-histories as a validity test

A useful way to evaluate an inference is to construct a counter-history: a plausible history that produces the observation but falsifies the claimed relation. Four counter-histories recur.

Common-predecessor ambiguity occurs when two applications inherit code from an earlier version or template. Pairwise similarity cannot establish which is the source. *Component contamination* occurs when a large shared library dominates the feature space and makes otherwise unrelated apps appear close. *authorized divergence* occurs when a legitimate maintainer refactors, obfuscates, localizes, or regenerates an application so heavily that its authorized successor looks dissimilar. *authenticated malice* occurs when a compromised or maliciously authorized pipeline produces a correctly signed artifact. These histories respectively challenge direction, attribution, recall, and security inferences.

Counter-histories do not prove a detector useless. They identify the assumptions under which its output is probative. A marketplace may reasonably combine similarity with publisher identity, upload time, signing history, and user reports. The resulting decision is a fusion system; it should not be described as if the similarity feature alone established plagiarism.

3.6 Evidence fusion and non-substitutability

Evidence fusion is useful when observations challenge different counter-histories under explicit joint assumptions. Code and UI resemblance can weaken a shared-library-only explanation; an authenticated build record can supply direction; transparency can expose inconsistent releases; and behavior analysis can test the exact output. These roles do not require statistical independence. If probabilistic scores are combined, however, dependencies among features and shared label errors must be modeled rather than counting correlated observations as independent corroboration.

For an artifact A, an assurance case can be expressed as a set of linked claims:

$$C(A) = \{C_{\text{comp}}, C_{\text{rel}}, C_{\text{prov}}, C_{\text{auth}}, C_{\text{beh}}\}.$$

The links are explicit bindings: a digest joins an attestation to an APK; a source revision joins a repository event to a build; a library version joins component detection to a vulnerability record; and a package signature joins an artifact to

a release identity. If any binding is missing, the layers can refer to different objects while appearing to support one conclusion.

This model frames the remaining sections. Android repackaging research is rich in L1–L3 evidence. Supply-chain systems are rich in L4–L5 evidence. Android program analysis contributes to L6. AI-mediated development changes the transformations and principals that must be represented across all levels.

4 Android Repackaging and Artifact-Level Evidence

Android repackaging research began from a concrete abuse pattern: obtain an application, modify it, re-sign it, and redistribute the result. The modification may add advertising, remove payment checks, graft malicious code, change branding, or camouflage an application in a market. This history makes repackaging a natural lineage problem, but it also creates a temptation to collapse several tasks into one label. Candidate retrieval, similarity measurement, pair classification, direction assignment, maliciousness assessment, and authorization judgment are separate stages.

The review by Li et al. documents 57 approaches and emphasizes the difficulty of reproducing and comparing their results [42]. That finding remains a useful organizing constraint. A survey of evidence should not ask only which feature a detector uses. It should ask what unit is compared, which transformations are modeled, how pairs are labeled, whether common libraries are removed, whether direction is part of the label, and whether the evaluation contains realistic negatives. Those choices determine the claim ceiling.

4.1 Code and structural resemblance

Early scalable systems represented applications through fingerprints, normalized code, or structural abstractions. DroidMOSS uses fuzzy hashing to detect repackaged applications in third-party markets [96]. Attack of the Clones and AnDarwin use program representations intended to scale beyond exact byte matching [13, 14]. Juxtapp detects code reuse across Android applications, and WuKong combines candidate filtering with more detailed comparison [26, 76]. These systems illustrate an enduring design pattern: a cheap first-stage index narrows the comparison set, then a more discriminating representation evaluates likely pairs. Earlier studies of application plagiarism and market-level reuse also show that the same observed reuse can correspond to copying, legitimate components, or product-family evolution [58, 63].

The strength of code evidence depends on what is normalized. Identifier renaming can be suppressed by ignoring names or hashing normalized instructions. Package restructuring can be reduced by comparing below the package level. Control-flow graphs and call relations preserve more structure than token bags, but can change under inlining, outlining, reflection, packing, or control-flow rewriting. CodeMatch and DAPASA use different abstractions: the former compares normalized code structures, whereas the latter classifies sensitive static call subgraphs [18, 22]. DAPASA's category-conditioned features and historical family/antivirus labels do not supply a pairwise copy-history oracle; reflection, encryption, and the age of its sensitive-API mapping limit the interpretation. SimiDroid adds explanations for similarities, which is important because a single score does not tell an analyst whether the match comes from application logic, a library, or boilerplate [41]. RepDetector compares symbolic output states of selected core methods, but bounds undetermined loops at five iterations, simplifies types and omits native calls [25]. Its known-group and successful-transformation evaluations are not complete-program equivalence proofs.

CLANdroid instead retrieves functionally related applications, evaluated through market recommendations and users' judgments of descriptions and screenshots [47]. That oracle is not a repackaging label.

Feature robustness and claim validity are related but distinct. A method robust to identifier renaming may improve recall for transformed derivatives, yet the same robustness can increase matches among independently built applications that use common frameworks. A graph abstraction can suppress irrelevant details, but an abstraction that is too coarse creates collisions. The right representation therefore depends on the relation of interest and the candidate population.

Several works explicitly pursue scale. Achieving Accuracy and Scalability Simultaneously, WuKong, MassVet, and large-market analyses use staged processing or compact features to operate over many applications [11, 12, 30, 76]. Scale changes the evidence problem. In a small curated pair set, every candidate may be known to have a related counterpart. In a market-scale setting, most pairs are negatives, labels are sparse, and a small false-positive rate can generate an unmanageable review queue. Reporting accuracy on balanced pairs is not enough to estimate this operational burden.

4.2 Code heterogeneity and piggybacking

Piggybacking narrows the lineage question by looking for a largely reused host with injected or changed code. The systematic study of malicious code grafting documents how code is added to existing applications [45]. Code-heterogeneity features exploit the intuition that grafted regions may differ from the surrounding program, while DAPASA focuses on sensitive subgraphs associated with piggybacked behavior [18, 74]. These signals can support a hypothesis that code was inserted, but heterogeneity is not unique to malicious grafting. Applications commonly combine first-party code, generated code, libraries, and modules produced by different teams.

A stronger piggybacking claim therefore needs at least two bindings: a relationship between the candidate and a plausible host, and evidence that the differing region corresponds to an added transformation rather than ordinary variation. Temporal market records, signing changes, known original–piggybacked pairs, or authenticated source histories can supply direction. Behavior analysis can then assess the inserted code. Without those bindings, a detector may accurately locate anomalous code while remaining uncertain about origin and intent.

This distinction also explains why malware classifiers are relevant but not interchangeable with lineage detectors. DREBIN and MaMaDroid infer maliciousness from application features and behavioral abstractions [4, 49]; FlowDroid supports precise information-flow analysis within an Android lifecycle model [5]. Their observations can strengthen L6 claims about a candidate. They do not establish that the candidate was copied from a specific application. Conversely, a near-perfect clone match does not establish that the changed or unchanged behavior is malicious.

4.3 Resources, user interfaces, and externally visible behavior

Code is only one layer of an Android application. Resources, layouts, images, strings, manifests, and user-interface structure can remain recognizable after code rewriting. Resource-driven systems exploit this fact [20, 67]. ViewDroid and DroidEagle use visual or view-level evidence to detect similar applications despite some code obfuscation [71, 90]. UI-clone evidence is especially useful for brand impersonation and applications that preserve a user-facing design while replacing implementation details.

Resource and UI evidence also has a clear claim ceiling. Common templates, design systems, icon packs, and generated layouts can create legitimate resemblance. Localization can make legitimate variants appear dissimilar, while a malicious clone can deliberately alter images and colors. Screenshot evidence depends on explored states, device configuration, rendering, and input sequences. A UI match is consequently strong evidence of presentation resemblance in the sampled states, not proof of source-code derivation or full functional equivalence.

Combining UI and code evidence can eliminate some alternative histories. A pair that shares distinctive first-party code and a distinctive layout is less likely to be explained by a common library alone. Nevertheless, independence

should not be assumed: generated UI bindings and resource identifiers can influence both code and view features. A fusion model should report which underlying artifacts contribute to each score.

4.4 Watermarks, deterrence, and owner-origin evidence

AppInk takes a different approach: embed evidence intended to survive transformation and support later ownership or repackaging investigation [95]. Its five-application watermark evaluation is bounded feasibility evidence, not a universal removal/forgery guarantee. A successfully verified watermark can be stronger than incidental similarity because the mark is designed and controlled by a party before distribution. It can help establish that a candidate contains material derived from a marked artifact.

Watermarks still depend on assumptions. The mark must be bound to an identity, created before the dispute, resistant to removal and forgery, and specific enough not to collide across applications. A verifier must distinguish knowledge of a mark from authorization to use the marked code. If a build service, contractor, or licensee legitimately receives a marked source base, the mark establishes origin but not the legal or policy status of every derivative. Watermarking therefore contributes to L3 when deployment and identity records support direction, and to L5 only when an external authorization policy is added.

Deterrence and detection should also be separated. A transformation that makes removal expensive can deter a repackager even if the detector is not perfect. Conversely, a highly detectable mark may be easy to strip once its location is known. Evaluation should include mark survival, false claims, collusion or copying, performance cost, and the risk that the protection itself changes downstream detector features.

4.5 Transformation models and adversarial validity

Android applications undergo ordinary and adversarial transformations: compiler changes, shrinking, optimization, identifier renaming, control-flow rewriting, string encryption, packing, library updates, resource replacement, and signing. DroidChameleon's historical stress test applied selected transformations to six old samples and ten anti-malware products chosen to detect the originals [60]. The lesson transfers to lineage analysis: a detector's transformation model is part of its claim, not an implementation detail. Finite transformations and detection changes do not by themselves establish present-day failure rates or complete runtime equivalence.

Transformation robustness should be described with a matrix rather than a binary "obfuscation resistant" label. A feature may survive renaming but fail under dead-code removal; survive resource edits but fail under package flattening; or tolerate local rewriting while failing when an application is regenerated from a high-level specification. CodeMatch, FSquaDRA, SimiDroid, and resource-based systems occupy different cells in this matrix [20, 22, 41, 94]. For the representations and artifact domains reviewed here, robustness is demonstrated only for the stated operator families; coarser abstractions can also match distinct artifacts. These results do not establish an impossibility for every representation and transformation model.

There is also a direction asymmetry. A detector optimized for finding transformed copies of a known original has a reference artifact and can compute expensive targeted features. Open-world clone discovery must compare many artifacts without knowing the ancestor. Evaluations should state which setting is measured. Results from reference-based detection do not automatically transfer to all-pairs market discovery.

Table 5. Representative Android artifact-evidence families. The claim ceiling assumes no auxiliary provenance or policy evidence.

Family and examples	Observation and modeled resilience	Ceiling	Recurring failure boundary
Fuzzy and normalized code [13, 96]	Method or application fingerprints; often tolerates renaming and local edits	L2	Common libraries, templates, coarse collisions, unseen rewrites
Graph and semantic structure [14, 18, 22]	Call, control, dependency, or sensitive subgraphs	L2–L3 with pair oracle	Reflection, graph rewriting, incomplete recovery, direction supplied by labels
Staged market-scale matching [11, 12, 30, 76]	Cheap candidate retrieval followed by detailed comparison	L2	Base-rate effects, sparse labels, candidate-stage false negatives
Resources and UI [20, 67, 71, 90]	Resource sets, layouts, images, or rendered views	L2	Templates, localization, state coverage, deliberate restyling
Embedded marks [95]	Designed watermark verified in a candidate	L3 with deployment record	Removal, forgery, licensed reuse, weak identity binding
Malware and flow evidence [4, 5, 49, 74]	Static features, behavioral abstractions, flows, anomalous regions	L6 for modeled property	Does not identify ancestor; analysis unsoundness or incomplete environment

4.6 Ground truth, datasets, and comparison validity

Ground truth is the most consequential part of a repackaging evaluation. A positive pair can be established through controlled transformations, developer records, market chronology, known source relationships, watermarking, or manual investigation. Each oracle answers a different question. Controlled transformations provide exact direction and known changes but may be less diverse than field modifications. Market-derived pairs improve realism but can contain uncertain direction and false labels. Malware-family labels are not clone labels. Shared certificates can indicate a publisher relation but are neither necessary nor sufficient for derivation.

The RePack benchmark was created to support detector comparison, but its labels are operational rather than authenticated histories [42]. Sections 2.1 and 5 select similar code under different signing certificates; Section 5.1 orders candidate pairs using DEX creation times. Section 7 warns that timestamps can be manipulated and an apparent original may itself be repackaged. The positive-pair comparison in Section 5.5 measures recall against this construction, not open-world false-positive performance. AndroZoo supplies large-scale application artifacts and metadata that enable broader sampling [3]. These resources address availability and scale, but dataset access does not by itself solve oracle construction. A benchmark should expose how each pair was formed, which artifact is considered original, what transformations are observed, which common components were removed, and which negative pairs are challenging.

Origin uncertainty also appears in the primary studies. DroidMOSS relies on an official-market-original assumption; AdRob warns that a shared certificate does not identify an owner; WuKong explicitly treats its direction heuristics as imperfect; and APPraiser's download-count/market-ID origin rule fails if the original is missing [21, 30, 76, 96]. The piggybacking study likewise admits that its apparent original can already be repackaged [45]. DroidKin's threshold-defined relatives additionally include apps sharing public debugging certificates rather than one author [24]. These are different operational oracles, not independently authenticated histories.

Huang et al.'s framework deliberately relaxes complete APK runnability for static-detector evaluation and reports conversion/type-inference failures [28]. Static acceptance is therefore distinct from a deployable, behavior-preserving

transformation. AndroZoo's checksums identify collected snapshots, while crawler interruptions and version replacement prevent a complete market history [3].

A useful negative set is not a random sample of unrelated applications. Random negatives are often easy and can overstate discrimination. Hard negatives include applications from the same developer, versions of one legitimate product, applications built from the same template, apps sharing large libraries, and independently implemented apps with similar UI or functionality. These cases test whether the detector confuses legitimate relatedness with unauthorized derivation.

Temporal leakage is another risk. If a detector's signature database, threshold, or feature selection uses later versions of an application, evaluation on earlier pairs can accidentally reveal the answer. Splitting by application family, developer, market, and time is often more informative than a random pair split. The same principle applies to AI-era artifacts, where near-duplicate generated outputs from one prompt or agent trajectory can cross train and test boundaries.

Comparison also requires a common unit. Some studies count application pairs, others count applications, clusters, methods, or detected regions. Precision among returned pairs cannot be compared directly with family classification accuracy or top-k retrieval recall. A report should state the candidate population, class balance, threshold-selection set, unresolved labels, and review effort. The framework of Huang et al. and the later RePack critique both motivate this discipline [28, 42].

4.7 What artifact analysis establishes

Android artifact analysis is indispensable because process provenance is frequently absent, incomplete, or controlled by a party under investigation. The literature provides mature techniques for retrieving related candidates, localizing shared regions, resisting selected transformations, and analyzing behavior. Its strongest general contribution is not a universal repackaging verdict. It is a set of observations that can support composition, resemblance, and, with an external directional oracle, derivation.

The principal unresolved issue is evidence binding. A detector output should identify the exact artifact digest, compared region, ignored components, transformation assumptions, threshold, and candidate universe. If direction or authorization is asserted, the report should name the additional source: time, source history, signing identity, watermark deployment, attestation, or policy. This reporting pattern makes a detector useful in a larger assurance case without asking it to prove what it does not observe.

5 Third-Party Libraries and Component Lineage

Third-party libraries complicate every Android lineage claim. They can dominate an application's code, create similarity among unrelated products, carry vulnerabilities across many packages, and change independently of first-party logic. Removing all library code is not a safe default: the piggybacking study locates injected hooks inside popular library code [45]. Filtering common-code noise and checking component integrity are different tasks. The evidence problem is therefore not simply "filter libraries." It is to identify component boundaries, names, versions, transformations, and use while preserving uncertainty.

The systematic review by Zhan et al. organizes 74 Android-library studies and distinguishes detection, security analysis, maintenance, privilege separation, and other tasks [89]. That taxonomy establishes that "library detection" contains several relations. A module detector identifies candidate regions; a family detector attributes a region to a library; a version detector selects a release; and a vulnerability analysis links the selected version or reachable code to a security record. Each stage can fail independently.

5.1 Module separation

An APK does not necessarily preserve the source project's module boundaries. Build tools merge bytecode, shrink unused classes, rewrite names, desugar language features, and package transitive dependencies. A library may be partially included, several libraries may share utility classes, and generated adapters may connect first-party and third-party code. Package names therefore provide useful hints but not a reliable boundary under obfuscation.

LibRadar, LibD, LibScout, LibID, and ORLIS illustrate different combinations of package structure, normalized method features, class dependencies, call graphs, and databases [6, 46, 48, 77, 91]. The design choice affects both recall and the object being reported. A detector that returns individual library classes can remain robust when dead-code elimination removes part of a component, but the result may not correspond to a complete upstream release. A detector that requires a coherent module can provide stronger attribution when it succeeds but may lose partial embeddings.

Module separation matters for repackaging because shared-library code can produce false similarity. Yet removing candidate libraries before matching can erase relevant evidence when the transformation occurs inside a library or when injected code is placed under a familiar namespace. A sound workflow retains both views: first-party resemblance with common components discounted, and component-aware resemblance that reports which libraries and versions account for the match.

5.2 Family and version attribution

Family attribution maps an in-app region to an upstream library identity. Version attribution is harder because adjacent releases can differ by few methods, while shrinking can remove precisely the distinguishing code. LibScout focuses on reliable library detection and security applications, OSSPolice combines library identification with license and known-risk analysis, and ATVHunter targets reliable version detection for vulnerability identification [6, 17, 87]. LibID and ORLIS explicitly address obfuscation resilience [77, 91].

A version result is conditional on the reference database. The detector can select only among indexed releases and may face forks, republished binaries, shaded dependencies, or vendor patches. Reporting a single exact version without alternatives can overstate the evidence. A better output is a candidate set with distinguishing and missing features, plus an "unknown or modified" state when no indexed version explains the observed region.

Transformation assumptions remain system-specific. ORLIS's class-level matching assumes preserved dependencies, prototypes, and hierarchy rather than arbitrary class splitting/merging or call changes [77]. LibPecker's selected recall oracle is derived from package-name containment, and its matching assumes preserved package hierarchy [92]. Thus, recall within that positive universe is not unbiased recall over all obfuscated libraries. Java-bytecode signatures also do not automatically transfer to native code or ahead-of-time compiled output.

5.3 Obfuscation and transformation resilience

The TPL review shows that resilience is multidimensional [89]. Identifier renaming is comparatively easy to suppress by excluding names. Package flattening defeats systems that use namespace hierarchy to recover modules. Dead-code elimination removes features and boundaries. Control-flow randomization changes graphs and instruction sequences. String encryption removes constants used as signatures. Reflection and dynamic loading hide call relations. No detector handles all transformations equally.

The comparative question should therefore be stated as a conditional: given transformation family M, reference corpus D, and inclusion fraction f, can the system identify family or version? A single "obfuscation" flag conceals

Table 6. Component-evidence tasks and representative systems.

Task and examples	Principal evidence	Ceiling	Important uncertainty
Library-region discovery [46, 48, 77]	Package structure, class relations, calls, normalized code	L1	Partial inclusion, merged modules, generated bridges
Family identification [6, 91, 92]	Signatures compared with an indexed corpus	L1	Forks, unknown libraries, signature collisions, database coverage
Version attribution [17, 87]	Version-discriminating code or graph features	L1	Shrinking, patches, republishing, indistinguishable adjacent releases
Obfuscation-resilient attribution [77, 91]	Normalized classes, dependencies and indexed profiles	L1	Preserved relations, partial code and reference coverage
Library-update studies [15, 64]	Bundled library versions and changes across app releases	L1	Release order alone does not establish derivation; component versions may remain unresolved
Vulnerability-source studies [78]	Attribution to app or library code with security observations	L1; L6 for observed behavior	Component presence, exposure, and exploitability need separate evidence
De-duplication and updating [40]	Library separation, compatibility, storage, and update effects	L1; L6 for evaluated behavior	Tested updates do not establish ancestry or behavior in other configurations
Potentially malicious libraries [93]	Library attribution plus security or behavior analysis	L1+L6	Attribution and maliciousness have separate oracles

too much. The "Are We There Yet?" evaluation directly questions the readiness and comparability of Android TPL detectors [88]. Its source also documents database-coverage differences, dataset choices, and implementation repairs [88]. Comparisons must expose these choices rather than attribute every score difference to the matching algorithm.

Generated code adds a related transformation. Code generators can copy templates, synthesize adapters, inline dependencies, or produce semantically similar but structurally novel implementations. A library detector may correctly report no known library even though the generated code implements a familiar component interface. Conversely, an agent may import a package based on a textual suggestion, making the dependency a direct supply-chain decision. TPL evidence should distinguish reused implementation from independently generated compatibility code.

5.4 Updates, vulnerabilities, and reachability

Library identification is often motivated by vulnerability management. Studies of mobile library updates show that developers do not always adopt new versions promptly, and that application vulnerabilities can originate in embedded third-party code [15, 64, 78]. AppCommune explores automated de-duplication and updating [40]. These studies connect composition evidence to maintenance behavior, but a vulnerability claim requires further steps. LibID identifies apps carrying a vulnerable OkHttp version that do not invoke the affected certificate-pinning method [91]. The origins study reports library-associated findings for 71–72% of vulnerable free apps and 46–52% of vulnerable paid apps; these are app-level fractions, not percentages of all flaws or confirmed exploitations [78]. Static reachability filtering still cannot establish actual use, and dynamic/native code remains outside its analysis.

First, the version must be identified with sufficient confidence. Second, the vulnerability record must apply to that version and configuration. Third, the affected code must be present after shrinking. Fourth, it may be necessary to

establish reachability from an application entry point. Fifth, environment and permissions can affect exploitability. A software bill of materials or detector output typically satisfies only the first part of this chain.

Update success is similarly bounded. KeepUpdated's API-compatibility analysis does not establish semantic preservation or absence of new bugs [15]. AppCommune's short tests triggered 63% of patched APIs, with subsequent volunteer use and checks for five selected vulnerabilities [40]. Observed normal runs are useful evidence, but not proof that every API-compatible update preserves behavior or has developer authorization.

This layered view prevents two opposite errors. Treating every detected vulnerable version as exploitable inflates risk; treating a library as harmless because an affected method was not observed can miss dynamically loaded or reflective paths. The evidence report should state whether it establishes presence, reachability, triggerability, or observed behavior.

5.5 License, ownership, and common-code ambiguity

OSSPolice demonstrates that component attribution can support license and security analyses [17]. The ownership inference remains policy dependent. Detecting open-source code in an APK does not establish a violation unless the applicable license, distribution obligations, modifications, and offered source are evaluated. Similarly, finding a proprietary SDK in two applications does not establish copying between the application developers.

This is the main connection to repackaging. Common components are alternative explanations for similarity. The first-party residual after component accounting is often more probative of direct relationship, but only if the component detector's uncertainty is retained. A hard filter can turn library false negatives into clone false positives and library false positives into clone false negatives. Joint models should propagate component probabilities or candidate sets instead of making an irreversible preprocessing decision.

The early investigation of common Android libraries identified how widespread reuse complicates application comparison [43]. Later systems improved family and version detection, but the conceptual lesson remains: composition is not ancestry. A correct component map narrows the space of plausible histories; it does not select one history without direction evidence.

5.6 A component-aware lineage record

A component-aware lineage record for an APK should contain: the APK digest; candidate first-party regions; detected library regions; family and version candidate sets; omitted or unknown regions; transformation assumptions; reference-database date; and links from each region to the evidence used for attribution. When vulnerability or license claims are made, the record should add reachability or policy evidence rather than silently promoting a composition result.

This record also helps connect artifact analysis to process provenance. An authenticated build record may declare dependencies, while binary analysis observes what was actually packaged. Agreement strengthens confidence. Disagreement is diagnostically useful: the build record may be incomplete, the binary detector may miss a transformed component, or the final artifact may not correspond to the claimed build. The two evidence sources should be reconciled, not treated as redundant.

6 Provenance, Release Authority, and Supply-Chain Integrity

Artifact analysis begins with the product and reasons backward. Supply-chain systems attempt to preserve evidence while the product is created and distributed. This shift changes what can be established. Instead of inferring a possible

history from similarity, a provenance system records principals, steps, materials, and products. The record can make direction and authorization explicit, but only within the policy and collection boundary.

Recent systematizations describe supply-chain defenses through properties, attack classes, and causal locations [29, 36, 55]. Their common lesson is that "the software supply chain" is not one service. It is a graph of source repositories, dependency registries, build workers, CI configuration, signing services, release metadata, mirrors, stores, and update clients. An Android APK sits near the end of that graph. A trustworthy store download therefore depends on more than the APK's own signature.

6.1 Repository metadata and delegated authority

The Update Framework separates repository responsibilities into roles and signs metadata describing targets, versions, expiration, and delegation [65]. The design addresses attacks in which an adversary compromises a key or repository and attempts to serve stale, substituted, or inconsistent updates. Diplomat extends the principle of delegation to community repositories [35], under trusted-developer and separately protected-key assumptions; its stronger offline policy can delay updates for new or infrequently updated projects. These systems show why authorization should be scoped. A key authorized to publish one target set should not automatically authorize every package or metadata function.

Mapped to Android, a release policy could distinguish the principal that approves source, the builder that produces an APK, the signer that applies a release key, and the store account that publishes the package. Existing Android signing binds an APK to a key and supports update continuity, but a cryptographic verification result does not independently establish the real-world identity or current authority of the key holder. Key rotation, delegation, organization changes, compromised accounts, and outsourced builds all require policy context.

Repository metadata also carries freshness. A correctly signed old release may be vulnerable or revoked. Rollback and freeze defenses therefore complement signature verification. This matters for mobile ecosystems in which alternative markets, enterprise distribution, archival sites, and device update behavior can expose different release states.

6.2 Step attestations and source-to-product binding

in-toto records authenticated link metadata for steps and checks the resulting supply chain against a layout defined by the project owner [75]. The important abstraction is not a particular file format; it is a signed relation among materials, a command or step, products, and a functionary. When the final product digest is an APK digest, the record can establish a directed source-to-package path that artifact similarity alone cannot.

A complete Android-oriented layout might include source checkout, dependency resolution, code generation, resource processing, compilation, shrinking or obfuscation, native build, packaging, signing, and store upload. Each step changes the evidence available later. For example, a code-generation step should record the generator or service identity, version, policy, prompt or specification reference, retrieved context, and output digest. A shrinking step should record configuration because it affects both library evidence and clone-detection features.

Attestations are conditional evidence. A valid record shows that a key asserted a step and that digests link inputs to outputs. It does not prove that the command was the only process executed, that the worker was uncompromised, that secrets were not exfiltrated, or that the output is benign. Trusted execution, hermetic builds, policy review, and independent analysis can reduce those risks. The attestation should never be described as a behavior proof.

6.3 Transparency and consistent views

Transparency mechanisms make release events publicly or collectively observable. CHAINIAC combines collectively signed skipchains, rebuild verification and approval under secure-bootstrap and honest-threshold assumptions [51]. Contour commits binary-publication records to a log; data retrieval additionally needs available archives, and code inspection remains a separate obligation [2]. The mechanisms do not turn a logged commitment into evidence of benign behavior.

Transparency addresses a failure mode that ordinary signatures do not: an authorized key can sign different artifacts for different observers. Logging releases and attestations enables monitors to detect inconsistent or omitted events, subject to log availability and monitoring. In an Android setting, a transparency record can bind package name, version, signing identity, APK digest, build provenance, and publication time. It does not decide whether two versions are semantically equivalent or whether a changed permission is appropriate; those remain analysis questions.

Legitimate instead binds a user's recorded intent to Git commit content and parent/merge fields through client verification and signed verification records [1]. Its security argument assumes independent trusted key management and admits that manual UI checks are imperfect. Such records do not themselves certify real-person identity, licensing, or benign changes.

Transparency can strengthen historical adjudication. If digest A is logged as a build product before candidate B appears in another market, localized resemblance and authenticated chronology support a possible $A \rightarrow B$ history. They do not exclude siblings derived from an earlier source, nor does a missing record for B establish copying. Direction requires a link to the transformation or another oracle that distinguishes these histories.

6.4 Reproducible builds and independent checking

Reproducible builds ask whether independent parties, given the same declared inputs and build procedure, obtain bit-for-bit identical outputs [37]. Reproducible Containers presents DetTrace, a deterministic Linux/x86-64 execution environment [39]. Its boundary includes the initial filesystem/configuration and well-behaved programs; unsupported cases and timeouts are not successful reproductions. This is not a guarantee for arbitrary container images or Android builds. For Android, reproducibility can bind a published source revision and dependency set to an APK digest without requiring trust in one builder. It is particularly valuable when release signing is centralized but build verification can be distributed.

Reproducibility is often overstated in two ways. First, equality of independent outputs is evidence that the declared procedure is deterministic and that those executions agree; it does not prove that the declared source is authorized or secure. Second, a failed reproduction is diagnostically ambiguous. The cause may be a malicious hidden input, an undeclared dependency, a timestamp, toolchain variation, environment leakage, or ordinary nondeterminism. The result should therefore be reported with input capture and difference localization.

Reproducibility and artifact similarity occupy adjacent but different levels. A reproducible build provides L0 identity for outputs and supports L4 when inputs and procedure are authenticated. A similarity detector can compare releases that are intentionally non-identical or generated under different toolchains. A mature pipeline uses reproducibility where exact equality is expected and transformation-aware comparison where it is not.

6.5 CI workflows and build-system attack surface

Continuous-integration workflows are executable policy. They select dependencies, obtain credentials, invoke third-party actions, build artifacts, and publish releases. The GitHub CI study analyzes historical bot-event/workflow observations from 2019–2021 [33]; its sampled configurations and commit-message vulnerability proxies are not present-day vulnerability certificates. A signed attestation emitted by a compromised workflow can faithfully record a malicious process. The trustworthiness of provenance therefore depends on who can modify workflow definitions, which actions are pinned, how secrets are scoped, and whether untrusted contributions can influence privileged steps.

The mobile build adds further attack surfaces: signing keys, store credentials, Gradle plugins, Android SDK components, native toolchains, and remote build caches. A cross-layer assurance case should treat the build configuration and dependency resolver as evidence-bearing artifacts. Their digests and authorization history are part of the lineage of the APK.

A practical implication is that provenance collection must begin before the final packaging step. Recording only the signer and APK digest misses source substitutions, malicious plugins, generated code, and dependency-resolution attacks. At the same time, collecting every event without a policy produces an uninterpretable log. The record should be designed around explicit claims and verification rules.

6.6 Package ecosystems and transitive risk

Package-manager research demonstrates that repositories and dependency ecosystems are active attack surfaces. Early work examined attacks on package managers themselves [10]. Measurements of interpreted-language repositories identified malicious packages and supply-chain attack patterns [16]. The Backstabber's Knife Collection cataloged real open-source supply-chain attacks [53]. Studies of npm found dense dependency relations, maintainer concentration, and weak links that allow risk to propagate beyond a directly selected package [85, 97].

These findings motivate Android-facing risk hypotheses, not a measured transfer rate between package ecosystems. A mobile application can depend on Maven artifacts, Gradle plugins, JavaScript packages in a hybrid layer, native binaries, model files, and hosted services. An application-level SBOM that lists only direct libraries can omit build-time plugins or generated artifacts. Conversely, listing every transitive package does not establish that its code reached the final APK. Process records and binary component evidence should be reconciled.

The measurements also have distinct oracles: npm reach describes potential impact, weak-link surveys report practitioner perceptions, and collected malicious-package cases are neither a prevalence census nor complete recall [16, 53, 85, 97]. Unflagged packages are not thereby certified benign.

Empirical SBOM research reinforces this distinction. Repository-discussion analysis identifies recurring design questions around what to include, how to identify components, and how producers and consumers exchange the resulting records [7]. Interviews and survey evidence from SBOM stakeholders report practical challenges spanning generation, maintenance, consumption, and incentives [69]. These studies support the need for explicit object and policy bindings; they do not show that an SBOM entry proves code inclusion, reachability, provenance, authorization, or absence of vulnerable behavior.

The distinction between dependency intent and packaged reality is crucial. A lockfile or resolver log says what the build intended or selected. Binary analysis says what appears in the output. Reachability analysis says what can execute. Runtime observation says what executed under sampled inputs. These are four different evidence layers.

Table 7. Supply-chain mechanisms and their Android lineage contribution.

Mechanism	Direct contribution	Ceiling	Residual question
Signed repository metadata and delegation [35, 65]	Target identity, freshness, version, and policy-scoped publishing roles	L5	Whether the target was correctly or safely built
Step attestations and layouts [75]	Authenticated relation among materials, functionary, step, and product	L4–L5	Completeness of capture and trust in execution
Transparency and collective histories [2, 51]	Auditable publication history and resistance to equivocation	L4/L5 with step/policy evidence	Logging alone proves neither build history nor release authority
Reproducible builds [37, 51]	Independent equality check for declared inputs and process	L0; L4 with authenticated inputs	Authorization and security of declared inputs
Signed Git intent records [1]	Commit-content and declared-intent binding	L4 under client/key assumptions	Manual confirmation and key-to-person identity
Deterministic execution [39]	Repeatable execution under declared initial state	Supports equality checking	Supported platform/program assumptions, missing input capture
CI-workflow analysis [33]	Historical risks in machinery that creates attestations and releases	Supports L4 validity	Hidden services, credential compromise, runtime environment
Package-ecosystem measurement [16, 53, 85, 97]	Empirical attack and dependency-risk evidence	L1+L6	Exact transfer to the packaged mobile artifact

6.7 The source-to-APK evidence chain

Figure 2 places the evidence families along a mobile lifecycle. The links are digest and identity bindings. A gap means that records on the two sides may describe different artifacts or principals.

A minimally useful chain records a source revision, resolved dependency set, build and generation configuration, output APK digest, signing identity, release authorization, and publication event. Artifact analysis can then verify the packaged components and compare the output with prior or disputed artifacts. Behavior analysis evaluates the exact digest. This structure does not guarantee security, but it makes disagreements localizable and claims reviewable.

7 AI-Mediated Transformations and Mobile Lineage

AI-assisted development is not one transformation. It ranges from completion of a few lines to generation of an application skeleton, selection of dependencies, modification across multiple files, test execution, debugging, and release preparation. The relevant provenance question is therefore not merely whether "AI was used." It is which artifact-changing decisions were delegated, which context influenced them, and how the resulting outputs were bound to the final application.

7.1 From text descriptions to mobile artifacts

Text2App translates natural-language descriptions into a Simplified App Representation (SAR), then uses predefined templates to generate MIT App Inventor Scheme and Blockly files [27]. Figure 2 and Section 3.2 describe packaging

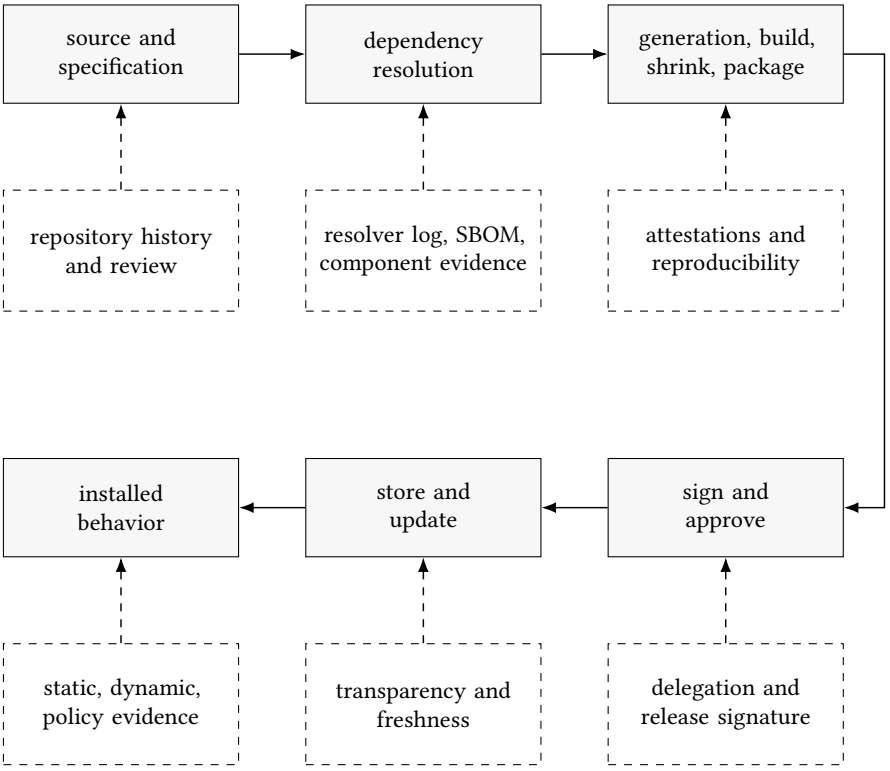


Fig. 2. Cross-layer evidence along an Android lifecycle. Every transition requires an object binding; evidence attached to one stage does not automatically validate another stage.

these files as an App Inventor project and compiling it through the App Inventor server to an APK. The component vocabulary is restricted; native-language app development is future work. This exposes a description–representation–project–APK chain, not unrestricted native Android synthesis. A generated application can share layout and behavior with other outputs while containing different source text. Conversely, two descriptions can produce similar source through templates or libraries.

AppForge is an Android development benchmark that evaluates generated projects and coding agents, including compilation-feedback refinement [59]. It supplies a functional evaluation setting, not an authenticated lineage system. Appendix B.3 defines functional success as compilation plus passage of all specified tests; crash rate is a separate metric, and functional success does not certify security or crash-free behavior. In a tool-using workflow, prompts, retrieved material, tool responses, repository state, and accepted revisions can all affect the final output. A final source diff does not record that complete path.

These systems do not make traditional repackaging evidence obsolete. A generated app can itself be copied, repackaged, or combined with existing code. What changes is the space of legitimate relatedness. Multiple authorized outputs may descend from the same prompt, template, or model-mediated plan without one being copied from another. Direction cannot be inferred from pairwise similarity alone.

7.2 Nondeterminism and reproducibility

Conventional build reproducibility assumes that a declared input set and deterministic procedure should yield the same bytes. Generative systems may sample different outputs for the same prompt, call external tools, or depend on changing model and retrieval services. Exact reproduction can therefore be unavailable even when the process is authorized.

This does not eliminate reproducibility as an objective; it changes what must be reproduced. One target is *record reproducibility*: an independent verifier can confirm the prompt or specification reference, model or service identity, parameters, tool sequence, retrieved materials, and accepted outputs. Another is *decision replay*: a recorded output is retained and subsequent deterministic build steps reproduce the APK. A third is *distributional replication*: repeated generations satisfy a defined property, which is an empirical model claim rather than artifact lineage.

For mobile supply-chain assurance, preserving the accepted generated source or patch is usually more important than reproducing the model's sampling event. The build attestation should treat that accepted output as a material with a digest. If policy requires regeneration, the record must acknowledge that regenerated code may be a sibling rather than an identical ancestor.

7.3 Security of generated code

Studies of code assistants show that generated suggestions can contain security weaknesses and that user interaction affects the result. Pearce et al. evaluated the 2021 Copilot technical preview on targeted security scenarios using CodeQL and manual assessment, without establishing functional correctness [56]. Perry et al.'s 47-participant study used codex-davinci-002 and found less secure assisted results on four of five analyzed tasks, with mixed C-task findings [57]. Sandoval et al. studied 58 student programmers implementing a C shopping-list API and reported a small security impact in that setting [66]. Khoury et al. examined 21 small ChatGPT-3.5 programs against selected attack classes; improvement prompts did not certify general security [31]. Sandoval's metric counts manually annotated CWEs per line of valid code, with a chosen 10% non-inferiority tolerance rather than a zero-harm criterion. The differing tasks, models, participants, and vulnerability oracles preclude a pooled claim that assistance invariably increases insecurity.

For lineage, the important finding is structural: generation provenance does not imply secure behavior. An authorized model-assisted change may introduce a vulnerability; an unauthorized derivative may remove one. Security review must analyze the actual accepted code and built artifact. The provenance record helps identify which model-mediated step introduced a change, but the vulnerability evidence comes from static analysis, testing, review, or formal reasoning.

Human approval also needs careful interpretation. Clicking "accept" establishes an interaction event, not necessarily informed review. A policy may require that a maintainer approve generated changes, but the evidential value depends on what was shown, which tests ran, whether warnings were visible, and whether the reviewer had authority for the affected component. Authorization and assurance remain separate.

7.4 Dependencies and package hallucination

Generative systems can recommend dependencies by name. Spracklen et al. measure nonexistent package recommendations in Python and JavaScript generation, using install-command and follow-up-prompt heuristics against PyPI/npm snapshots [68]. Their registry-abuse scenario motivates a dependency risk, but it does not measure an Android attack rate. Transfer to Maven coordinates, Gradle plugins, native packages or download URLs is a hypothesis requiring ecosystem-specific validation; JavaScript dependencies in hybrid apps are closer to the measured setting. The suggestion may enter source or build configuration even when no copied code appears in the model output.

This risk links AI assistance directly to supply-chain provenance. A robust record should capture the dependency request, resolver result, repository origin, version, integrity digest, and whether the dependency was already approved. A policy can reject unrecognized repositories or require lockfile review. Binary component analysis then checks whether the resolved material appears in the APK. Neither the prompt log nor the binary detector alone provides the full chain.

Package hallucination also motivates a counter-history for library attribution. A newly registered malicious package may intentionally implement an interface or name suggested by a model. The package's code is not necessarily similar to an established library, so clone detection may not flag it. Repository provenance, registration chronology, maintainer identity, and dependency-policy evidence become central.

7.5 Training-data and privacy lineage

CodexLeaks identifies sensitive-output candidates through public-code search and author review [52]. Its 43 validated leakage responses among 2,560 responses are observations under that pipeline, not authenticated membership in an unavailable training corpus. The resulting lineage question differs from ordinary source copying: output association does not certify the exact training example or a directed repository-copy event. Sensitive material in a final APK is therefore a risk hypothesis requiring artifact-specific investigation, not an outcome measured by this study.

Artifact similarity can still help when a suspected source is known, but absence of a match is weak evidence because generation may transform names and structure. Provenance records can establish which model and prompt produced the accepted output, yet they may not reveal the relevant training example. Policy and legal conclusions require evidence beyond technical resemblance, including license, confidentiality, and permitted-use analysis.

Privacy also affects what provenance should retain. Storing complete prompts, retrieved files, and model outputs may itself expose secrets. A mobile-development provenance design should support selective disclosure: retain cryptographic commitments, access-controlled detailed records, and public metadata sufficient to bind the final artifact without publishing sensitive context. The record needs an evidence-retention policy, not indiscriminate logging.

7.6 Multi-principal and tool-mediated authorship

An agentic workflow can include a human requester, model provider, model instance, retrieval source, code repository, build service, package registry, test system, reviewer, and release signer. These are not equivalent principals. The model service may produce candidate text but lack release authority. A human may approve a patch but not control the signing key. A build service may emit an attestation but not own the source.

Supply-chain layouts are well suited to representing these roles when model-mediated steps are treated as explicit transformations. The layout can require that generated outputs pass tests, static checks, dependency policy, and human review before signing. It can also distinguish advisory output from accepted material. This is more precise than adding one boolean "AI-generated" field to the APK metadata.

The same role separation helps incident response. If a vulnerability is found, investigators can ask whether it originated in a generated patch, retrieved example, dependency recommendation, manual edit, or build-time transformation. The answer guides remediation and model or policy changes. Without step records, the final Git history may collapse several causes into one commit.

7.7 A minimum evidence record for generated transformations

A generated transformation record should identify: the parent source revision; task or specification identifier; model or service identity at the granularity available; generation parameters; tool and retrieval accesses; candidate-output

Table 8. AI-mediated development risks and the evidence needed to reason about them.

Risk or ambiguity	Why final-artifact similarity is insufficient	Complementary evidence	Main claim
Sibling generations (analytical scenario) [27, 59]	Similarity does not reveal whether one output copied another	Specification binding, generation records, accepted-output digests	L3–L4
Insecure generated code [31, 56, 57, 66]	Origin says nothing about security of accepted behavior	Static/dynamic analysis, tests, review evidence, exact artifact binding	L6
Hallucinated dependencies [68]	Malicious package may be novel and not resemble a known library	Resolver provenance, repository identity, lockfile, component verification	L1+L4
Sensitive-output association [52]	Exact training membership and directed ancestry are not authenticated	Model/prompt record, public-source investigation, access-controlled context	Disclosure risk; L3 unresolved
Agentic multi-file edits [59]	A final diff alone need not preserve the intermediate history	Step attestations, tool transcripts, policy checkpoints	L4–L5
Nondeterministic regeneration	Exact output may not be reproducible from a prompt alone	Preserve accepted output; separate record replay from model replication	L0+L4

digests; the accepted patch or file digests; tests and analyses run; human or policy approvals; and the downstream build product. Sensitive context can be access controlled, but the public or shared record should still bind the accepted output to the final APK.

This record serves three purposes. It establishes direction from a parent revision to an accepted generated change. It separates generation from approval and release. It gives artifact and behavior analyses exact objects to examine. The record does not prove that the model never used unrecorded training material or that the accepted code is secure. It makes those residual questions explicit rather than hiding them behind a generic provenance claim.

8 Cross-Layer Synthesis: What the Evidence Can Establish

The preceding literatures solve different parts of one problem. Android comparison methods observe the final artifact and recover relations that process records may omit. Component analysis separates first-party logic from dependencies and identifies versions. Supply-chain mechanisms record and authorize production steps. Behavioral analyses evaluate security and policy properties. AI-mediated workflows add transformations whose context may not be represented in source history. The synthesis question is how these observations combine without allowing one layer to stand in for another.

8.1 Six invalid substitutions

Six recurring substitutions expose the missing premise between available evidence and the operationally desired claim:

(1) **Similarity for direction.** A match cannot by itself distinguish $A \to B$, $B \to A$, or a common predecessor. Chronology and ownership can contribute evidence, but a directional oracle must distinguish these histories; an authenticated parent–transformation relation can supply that binding.
(2) **Signer for authorized publisher.** A signature binds an artifact to a key. Authorization additionally binds that key to a scoped role, validity interval, and policy; delegation and transparency systems provide such context [35, 65].
(3) **Provenance for benign behavior.** An attestation can bind declared inputs and a build to a digest while faithfully recording a vulnerable or malicious result. The exact artifact still requires behavior-specific analysis.

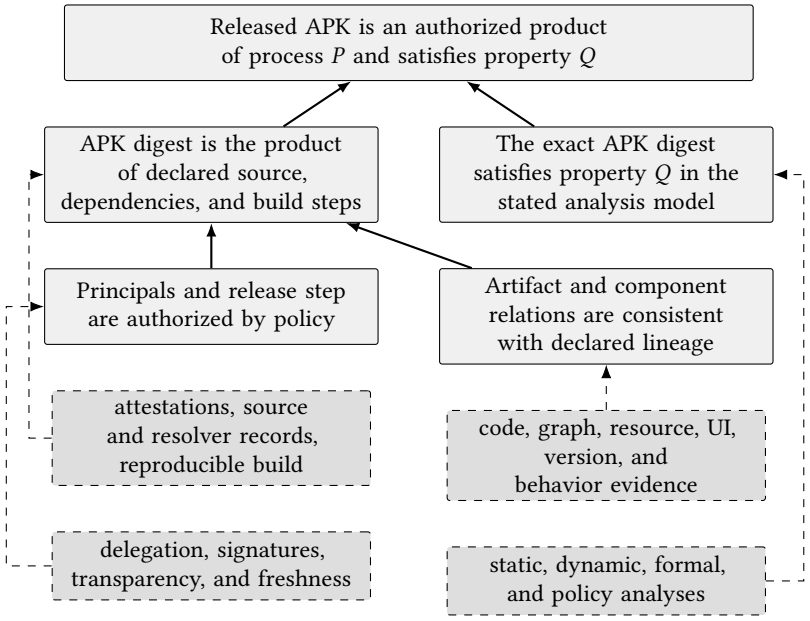


Fig. 3. Cross-layer assurance pattern. Solid arrows are subclaims; dashed arrows are evidence. Every evidence item must bind to the same artifact digest or to a documented predecessor relation.

(4) **Component presence for exploitability.** A family, version, or SBOM match does not establish that affected code is present, reachable, configured, and exposed; non-detection also does not prove absence after shading, shrinking, patching, or dynamic loading.
(5) **Reproducibility for approval.** Independent builders can reproduce an unauthorized output from unauthorized declared inputs. Reproducibility strengthens equality and source-to-binary binding, not input policy.
(6) **Generation log for complete causal history.** Prompts and responses may omit retrieval, tool state, rejected candidates, manual edits, or downstream transforms. The record must bind the accepted output, parent, and final artifact at the granularity required by the claim.

These are evaluation errors, not merely wording errors. An oracle can already encode direction, an honest-functionary prototype can hide compromised behavior, and full upstream packages can make versioning appear easier than partially packaged code. Reporting the missing premise identifies the hard negatives a study must include.

8.2 A cross-layer assurance pattern

Figure 3 presents an Android-focused assurance pattern building on existing artifact/operation/actor properties and supply-chain frameworks [29, 55, 79]. It is not a mandatory architecture. It is a claim graph that identifies the minimum bindings needed when an analyst wants to assert that a released Android artifact is an authorized product of a declared process and satisfies a stated property.

The top claim has two logically distinct branches. The provenance branch asks whether the APK is the product of the declared process and whether that process was authorized. The behavior branch asks whether the exact APK

satisfies property Q. Artifact and component analyses cross-check the declared lineage rather than merely repeating it. A mismatch does not automatically select which source is wrong; it triggers investigation.

This pattern accommodates incomplete provenance. If no trustworthy process record exists, artifact comparison may provide the best available evidence of relationship, but the top claim must be weakened. If the build is fully attested but behavior analysis is absent, the lineage claim can be strong while the security claim remains open. Reporting partial assurance is more informative than forcing a binary verdict.

8.3 Binding rules

Cross-layer evidence is useful only when records refer to the same objects. Four binding rules are therefore central.

(1) **Digest binding.** Every attestation, signature, store record, component report, and behavior result should name the artifact digest it covers. Package names and version strings are insufficient because they can be reused or equivocated.
(2) **Predecessor binding.** When artifacts are intentionally non-identical, the record should name the transformation and parent digest. A similarity score alone should not be used as the link when a process record can supply one.
(3) **Principal binding.** Keys, accounts, model services, builders, and reviewers should be mapped to scoped roles and validity intervals. "Signed" is not a complete identity statement.
(4) **Policy binding.** Authorization and behavior conclusions should name the policy version or property definition. A decision can change when the policy changes even if the artifact does not.

These rules also improve reproducibility. A later reviewer can determine whether two analyses disagree because they used different APKs, different component databases, different policies, or different environments. Without the bindings, disagreement is difficult to localize.

8.4 Contradiction patterns

A synthesis should preserve contradictions rather than averaging them away. Five patterns are particularly important.

Incompatible denominators. One study may report the fraction of known positive pairs detected, another the fraction of returned pairs judged positive, and a third classification accuracy on balanced data. The numbers cannot be ranked without reconstructing the denominators. This issue is prominent in repackaging evaluations and motivates the RePack effort [42].

Different transformation distributions. A method may appear robust because its benchmark contains renaming and packing but not control-flow restructuring or resource replacement. Another may target a different set. The correct synthesis is a coverage matrix, not a single robustness ordering.

Different objects. A library detector can report classes, modules, families, or versions. A clone detector can report methods, applications, pairs, or clusters. A supply-chain system can report steps, targets, or releases. Claims using different objects should not be counted as direct replications.

Different oracle provenance. Controlled transformations provide precise lineage labels but limited realism. Market-derived labels provide realism but uncertain ground truth. Authenticated process records provide direct history for participating projects but may not exist for adversarial artifacts. Each oracle closes one gap and opens another.

Table 9. Contradiction patterns and the information needed to resolve or bound them.

Observed disagreement	Likely hidden difference	Required report
Different accuracy or recall	Candidate universe, class balance, threshold, pair construction	Counts by unit, full confusion matrix or retrieval curve, threshold-selection procedure
Different obfuscation robustness	Transformation operators, strength, order, and composition	Transformation matrix and per-operator results
Different library-version results	Database coverage, module granularity, shrinkage, forks	Candidate versions, unknown state, observed/missing discriminators
Attestation and binary scan disagree	Undeclared dependency, detector miss, post-build change, wrong digest	Exact materials, product digest, detector database, packaging step
Signature valid but release disputed	Key compromise, expired role, wrong scope, account takeover	Delegation chain, validity interval, transparency and revocation state
Generated output cannot be reproduced	Nondeterminism, model drift, retrieval change, missing tool state	Accepted-output digest, model/service record, tool transcript, replay boundary

Different threat models. A signature scheme can be correct when keys are uncompromised; a delegation system may tolerate some compromised roles; a transparency system targets equivocation; a detector targets transformed copies; and a behavior analysis targets malicious functionality. Calling all of them "supply-chain security" does not make their guarantees interchangeable.

Table 9 turns these patterns into reporting obligations.

8.5 An evaluation blueprint

A cross-layer evaluation should be organized around falsification. It must name the object and proposition; retain positive, hard-negative, and unknown cases; separate candidate retrieval from verification; include ordered transformation compositions; split by family, developer, market, and time; and recheck a sample of all label states. For generated artifacts, outputs from one prompt family, seed project, or trajectory belong in one partition. For component and vulnerability claims, presence, exact version, reachability, and observed exploit conditions remain separate.

Operational reporting must include candidate volume, abstention, explanation size, and adjudication effort. Every downstream component, behavior, provenance, or authorization result must bind to the same APK digest or a documented predecessor relation. Table 10 turns these requirements into a minimum report; it is a floor rather than a complete domain-specific protocol.

8.6 Evidence profiles instead of universal rankings

The synthesis suggests reporting an evidence profile for each system. A profile is a vector over object granularity, claim level, transformation coverage, oracle strength, corpus transfer, and explanation. It avoids a universal ranking that would compare unlike goals. A fast UI detector may be the best first-stage signal for brand impersonation even if it does not establish source derivation. A cryptographic attestation may be decisive for an enrolled build while being irrelevant to an uncooperative third-party clone. A static analyzer may provide the strongest available privacy evidence while saying nothing about authorship.

Evidence profiles also expose combinations. A deployment can select complementary systems whose failure boundaries differ. For example, component-aware code matching can suppress common-library false positives; a release log can supply chronology, while direction still needs a parent or transformation binding; a delegated signature can provide

Table 10. Minimum reporting checklist for evidence-centered Android lineage evaluation.

Area	Required information	Failure prevented
Claim	Object, relation, claim level, assumptions	Similarity or signature promoted to a stronger claim
Corpus	Source, dates, markets, counts by unit, inclusion and unknowns	Hidden base rates and irreproducible denominators
Oracle	Label construction, direction source, recheck procedure	Circular evaluation and false certainty
Transformations	Operators, parameters, composition, ordinary vs adversarial	Vague robustness claims
Components	Library handling, version database, partial/unknown state	Common-code false positives and overconfident versioning
Provenance	Materials, steps, principals, product digests, layout or policy	Records that cannot be linked to the analyzed APK
Authorization	Role scope, validity, delegation, revocation, transparency	Treating any valid key as rightful publisher
Behavior	Property, tool/model, environment, coverage, limitations	Treating provenance as security assurance
AI mediation	Accepted-output digest, model/service, context boundary, tools, review	Unreplayable or misleading generation history
Operations	Candidate volume, latency, explanation and human review cost	Laboratory metrics that do not scale to deployment

authorization; and static/dynamic checks can evaluate behavior. The combination should be tested end to end because errors propagate: a wrong component boundary changes the similarity score, a wrong digest breaks provenance binding, and a stale policy changes authorization.

8.7 What the combined literature supports

The checked mechanisms motivate four organizing conclusions; the full 97-record synthesis has not been substantively verified. First, multiple Android studies leave origin assignment dependent on market, certificate, timestamp or reference-presence heuristics [21, 30, 42, 45, 76, 96]. Second, library family, version, invocation and reachability are distinct propositions, and library filtering can erase integrity evidence [45, 78, 89, 91]. Third, authenticated steps, delegated updates, transparent publication and reproducibility close different obligations under key, policy, initial-state and monitoring assumptions [2, 35, 39, 51, 65, 75]. Fourth, historical assistant studies measure task-specific security or output association, while mobile generation benchmarks measure specified functional tests; neither observation authenticates lineage or establishes universal safety [52, 56, 57, 59, 66].

The checked evidence provides no basis for treating one detector, signature, SBOM, attestation, or AI-disclosure field as a complete mobile-lineage solution. It supports a layered case in which each observation is limited to the proposition it measures and the cross-layer bindings are evaluated explicitly.

9 Research Agenda and Conclusion

The agenda shifts from a universal repackaging score to reviewable claims about artifact relations, production history, authority, and behavior. Detector evaluations should state the conclusion enabled and the premise supplied elsewhere.

9.1 Direction-labeled, transformation-complete benchmarks

Benchmarks should distinguish direct copy, common predecessor, shared library or template, legitimate version, independent generation, and authorized variant. They should record parent/child digests, ordered transformations, first-party/component regions, signing identities, chronology, and label provenance, while separating exact-history controlled cases from realistic field cases and retaining an unknown state.

Android pipelines combine shrinking, desugaring, generation, resource processing, cross-platform compilation, native code, and protection. Because order changes observability, lawful benchmarks should retain per-stage artifacts and report which relation survives each composition.

9.2 Provenance-to-APK and component-to-behavior binding

An Android provenance profile should terminate at the analyzed APK, binding source revision, resolved dependencies, generation/build configuration, builder, signer, publication event, and digest. Sensitive details may remain controlled, but commitments and predecessor relations must be checkable.

Declared composition should be reconciled with the binary. Reports must separate family, candidate version, partial inclusion, modification, reachability, and unknowns; reproducible builds, binary comparison, and transparency should act as independent checks rather than repeated declarations.

9.3 Auditable generated transformations

AI-mediated development needs artifact-level provenance. Each accepted patch or file set should bind a parent revision to a downstream product and record the model/service boundary, material retrieval/tool context, review decision, output digest, and unavailable evidence. Research must determine how to commit to sensitive context, retain sufficient agent history, cluster sibling generations without asserting copying, verify package recommendations, and represent model or tool drift when replay is impossible.

9.4 Open-world, temporal, and uncertainty-aware evaluation

Relatives, developers, libraries, markets, prompt families, and agent trajectories must not leak across random splits. Evaluations should hold out time and entity families, report candidate volume and abstention, keep unknown histories unknown, and distinguish absent, undetected, out-of-scope, and unavailable evidence.

Because keys, roles, packages, policies, and vulnerabilities change, decisions should be reproducible at a stated time and revisable through append-only records and explicit policy versions.

9.5 Decision-centered assurance tooling

Tooling should ingest artifact, component, provenance, signature, transparency, policy, and behavior evidence; check digest/predecessor bindings; and report supported, contradicted, and unresolved subclaims rather than one trust score.

Evaluation should follow retrieval, localization, adjudication, policy action, and appeal. With candidate population, policy, time, and review budget fixed, comparisons should test artifact-only, process-only, and bound cross-layer evidence and report precision/recall, abstention, reviewer effort, binding success, and localized disagreements. A decision log should also preserve the evidence available at the time, the policy version, the reason for abstention or escalation, and any later evidence that changed the outcome. Such revision tests whether the system supports accountable correction rather than only one-shot classification.

9.6 Stewardship and practice

AndroZoo and RePack show the value of shared data [3, 42]. Resources should preserve label provenance, transformations, licenses, errors, and historical denominators; constrained evaluations can separate public synthetic, redistributable open-source, metadata-only field, and controlled-access strata.

Stores should separate retrieval, lineage, authorization, and behavior review. Developers should retain source-to-APK/dependency bindings, protect signing roles, preserve analyzed artifacts, and record accepted generated changes. Researchers should report the claim ceiling rather than overstate direction or authority.

9.7 Conclusion

Android lineage comprises identity, composition, resemblance, derivation, process, authority, and behavior. Repackaging, TPL, supply-chain, and AI-development research illuminate different links; none should be promoted beyond its observation.

A defensible assurance case binds source and dependencies to build steps, steps to an APK digest, the digest to an authorized transparent release, and the exact artifact to component and behavior analyses. Artifact comparison remains essential when records are absent or disputed; provenance supplies direction and accountability. This claim-centered structure is a stronger target than a universal repackaging score.

Code availability. Code and accompanying materials are available in the project repository.

References

[1] Hammad Afzali, Santiago Torres-Arias, Reza Curtmola, and Justin Cappos. 2018. Le-git-imate: Towards Verifiable Web-Based Git Repositories. In *ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 469–482. doi:10.1145/3196494.3196523

[2] Mustafa Al-Bassam and Sarah Meiklejohn. 2018. Contour: A Practical System for Binary Transparency. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology (Lecture Notes in Computer Science, Vol. 11025)*. Springer, Cham, Switzerland, 94–110. doi:10.1007/978-3-030-00305-0_8

[3] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *IEEE/ACM International Conference on Mining Software Repositories*. IEEE, Piscataway, NJ, USA, 468–471. doi:10.1145/2901739.2903508

[4] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, and Konrad Rieck. 2014. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In *Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 15 pages.

[5] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*. Association for Computing Machinery, New York, NY, USA, 259–269. doi:10.1145/2594291.2594299

[6] Michael Backes, Sven Bugiel, and Erik Derr. 2016. Reliable Third-Party Library Detection in Android and Its Security Applications. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 356–367. doi:10.1145/2976749.2978333

[7] Tingting Bi, Boming Xia, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2024. On the Way to SBOMs: Investigating Design Issues and Solutions in Practice. *ACM Transactions on Software Engineering and Methodology* 33, 6, Article 149 (2024), 25 pages. doi:10.1145/3654442

[8] Stephen M. Blackburn, Kathryn S. McKinley, Robin Garner, Chris Hoffmann, Asjad M. Khan, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony L. Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanovic, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. 2008. Wake Up and Smell the Coffee: Evaluation Methodology for the 21st Century. *Commun. ACM* 51, 8 (2008), 83–89. doi:10.1145/1378704.1378723

[9] Pearl Brereton, Barbara A. Kitchenham, David Budgen, Mark Turner, and Mohamed Khalil. 2007. Lessons from Applying the Systematic Literature Review Process within the Software Engineering Domain. *Journal of Systems and Software* 80, 4 (2007), 571–583. doi:10.1016/j.jss.2006.07.009

[10] Justin Cappos, Justin Samuel, Scott Baker, and John H. Hartman. 2008. A Look in the Mirror: Attacks on Package Managers. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 565–574. doi:10.1145/1455770.1455841

[11] Kai Chen, Peng Liu, and Yingjun Zhang. 2014. Achieving Accuracy and Scalability Simultaneously in Detecting Application Clones on Android Markets. In *International Conference on Software Engineering*. IEEE, Piscataway, NJ, USA, 175–186. doi:10.1145/2568225.2568286

[12] Kai Chen, Peng Wang, Yeonjoon Lee, Xiaofeng Wang, Nan Zhang, Heqing Huang, Wei Zou, and Peng Liu. 2015. Finding Unknown Malice in 10 Seconds: Mass Vetting for New Threats at the Google-Play Scale. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 659–674.

[13] Jonathan Crussell, Clint Gibler, and Hao Chen. 2012. Attack of the Clones: Detecting Cloned Applications on Android Markets. In *European Symposium on Research in Computer Security*. Springer, Cham, Switzerland, 37–54. doi:10.1007/978-3-642-33167-1_3

[14] Jonathan Crussell, Clint Gibler, and Hao Chen. 2015. Andarwin: Scalable Detection of Android Application Clones Based on Semantics. *IEEE Transactions on Mobile Computing* 14, 10 (2015), 2007–2019. doi:10.1109/TMC.2014.2381212

[15] Erik Derr, Sven Bugiel, Sascha Fahl, Yasemin Acar, and Michael Backes. 2017. Keep Me Updated: An Empirical Study of Third-Party Library Updatability on Android. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2187–2200. doi:10.1145/3133956.3134059

[16] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 18 pages. doi:10.14722/ndss.2021.23055

[17] Ruian Duan, Ashish Bijlani, Meng Xu, Taesoo Kim, and Wenke Lee. 2017. Identifying Open-Source License Violation and 1-Day Security Risk at Large Scale. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2169–2185. doi:10.1145/3133956.3134048

[18] Ming Fan, Jun Liu, Wei Wang, Haifei Li, Zhenzhou Tian, and Ting Liu. 2017. DAPASA: Detecting Android Piggybacked Apps through Sensitive Subgraph Analysis. *IEEE Transactions on Information Forensics and Security* 12, 8 (2017), 1772–1785. doi:10.1109/TIFS.2017.2687880

[19] Parvez Faruki, Vijay Laxmi, Ammar Bharmal, Manoj Singh Gaur, and Vijay Ganmoor. 2015. AndroSimilar: Robust Signature for Detecting Variants of Android Malware. *Journal of Information Security and Applications* 22 (2015), 66–80. doi:10.1016/j.jisa.2014.10.011

[20] Olga Gadyatskaya, Andra-Lidia Lezza, and Yury Zhauniarovich. 2016. Evaluation of Resource-Based App Repackaging Detection in Android. In *Nordic Conference on Secure IT Systems*. Springer, Cham, Switzerland, 135–151. doi:10.1007/978-3-319-47560-8_9

[21] Clint Gibler, Ryan Stevens, Jonathan Crussell, Hao Chen, Hui Zang, and Heesook Choi. 2013. AdRob: Examining the Landscape and Impact of Android Application Plagiarism. In *International Conference on Mobile Systems, Applications, and Services*. Association for Computing Machinery, New York, NY, USA, 431–444. doi:10.1145/2462456.2464461

[22] Leonid Glanz, Sven Amann, Michael Eichberg, Michael Reif, Ben Hermann, Johannes Lerch, and Mira Mezini. 2017. CodeMatch: Obfuscation Won't Conceal Your Repackaged App. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 638–648. doi:10.1145/3106237.3106305

[23] Betul Gokkaya, Leonardo Aniello, and Basel Halak. 2026. Software Supply Chain: A Taxonomy of Attacks, Mitigations and Risk Assessment Strategies. *Journal of Information Security and Applications* 97 (2026), 104324. doi:10.1016/j.jisa.2025.104324

[24] Hugo Gonzalez, Natalia Stakhanova, and Ali A. Ghorbani. 2014. DroidKin: Lightweight Detection of Android Apps Similarity. In *International Conference on Security and Privacy in Communication Systems*. Springer, Cham, Switzerland, 436–453. doi:10.1007/978-3-319-23829-6_30

[25] Quanlong Guan, Heqing Huang, Weiqi Luo, and Sencun Zhu. 2016. Semantics-Based Repackaging Detection for Mobile Apps. In *International Symposium on Engineering Secure Software and Systems*. Springer, Cham, Switzerland, 89–105. doi:10.1007/978-3-319-30806-7_6

[26] Steve Hanna, Ling Huang, Edward Wu, Saung Li, Charles Chen, and Dawn Song. 2012. Juxtapp: A Scalable System for Detecting Code Reuse among Android Applications. In *Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, Cham, Switzerland, 62–81. doi:10.1007/978-3-642-37300-8_4

[27] Masum Hasan, Kazi Sajeed Mehrab, Wasi Uddin Ahmad, and Rifat Shahriyar. 2021. Text2App: A Framework for Creating Android Apps from Text Descriptions. arXiv:2104.08301. Presented in the NLP4Prog 2021 workshop program; no archival proceedings record was verified.

[28] Heqing Huang, Sencun Zhu, Peng Liu, and Dinghao Wu. 2013. A Framework for Evaluating Mobile App Repackaging Detection Algorithms. In *International Conference on Trust and Trustworthy Computing*. Springer, Cham, Switzerland, 169–186. doi:10.1007/978-3-642-38908-5_13

[29] Eman Abu Ishgair, Juanita Gomez, Marcela S. Melara, Alvaro Cardenas, and Santiago Torres-Arias. 2026. SoK: A Defense-Oriented Evaluation of Software Supply Chain Security. In *2026 IEEE 11th European Symposium on Security and Privacy (EuroS&P)*. IEEE, Piscataway, NJ, USA, 836–859. doi:10.1109/EuroSP68448.2026.00059

[30] Yuta Ishii, Takuya Watanabe, Mitsuaki Akiyama, and Tatsuya Mori. 2017. Appraiser: A Large Scale Analysis of Android Clone Apps. *IEICE Transactions on Information and Systems* E100-D, 8 (2017), 1703–1713. doi:10.1587/transinf.2016ICP0012

[31] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure Is Code Generated by ChatGPT?. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, Piscataway, NJ, USA, 2445–2451. doi:10.1109/SMC53992.2023.10394237

[32] Barbara Kitchenham. 2004. *Procedures for Performing Systematic Reviews*. Technical Report. Keele University.

[33] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the Security of GitHub CI Workflows. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 2747–2763.

[34] Pingfan Kong, Li Li, Jun Gao, Kui Liu, Tegawendé F. Bissyandé, and Jacques Klein. 2019. Automated Testing of Android Apps: A Systematic Literature Review. *IEEE Transactions on Reliability* 68, 1 (2019), 45–66. doi:10.1109/TR.2018.2865733

[35] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. 2016. Diplomat: Using Delegations to Protect Community Repositories. In *USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, Berkeley, CA, USA, 567–581.

[36] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *IEEE Symposium on Security and Privacy*. IEEE, Piscataway, NJ, USA, 1509–1526. doi:10.1109/SP46215.2023.10179304

[37] Chris Lamb and Stefano Zacchiroli. 2022. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software* 39, 2 (2022), 62–70. doi:10.1109/MS.2021.3073045

[38] Aikaterina Latsiou and Costas Lambrinoudakis. 2026. Cyber Supply Chain Risk Management: From Threats to Treatment. *International Journal of Information Security* 25, 1, Article 40 (2026), 11 pages. doi:10.1007/s10207-025-01207-9

[39] Omar S. Navarro Leija, Kelly Shiptoski, Ryan G. Scott, Baojun Wang, Nicholas Renner, Ryan R. Newton, and Joseph Devietti. 2020. Reproducible Containers. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 167–182. doi:10.1145/3373376.3378519

[40] Bowen Li, Yiming Zhang, Jing Li, Ruitao Feng, and Dawu Gu. 2019. AppCommune: Automated Third-Party Libraries De-Duplicating and Updating for Android Apps. In *IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Piscataway, NJ, USA, 344–354. doi:10.1109/SANER.2019.8668009

[41] Li Li, Tegawendé F. Bissyandé, and Jacques Klein. 2017. SimiDroid: Identifying and Explaining Similarities in Android Apps. In *IEEE International Conference on Trust, Security and Privacy in Computing and Communications*. IEEE, Piscataway, NJ, USA, 136–143. doi:10.1109/Trustcom/BigDataSE/ICESS.2017.230

[42] Li Li, Tegawendé F. Bissyandé, and Jacques Klein. 2021. Rebooting Research on Detecting Repackaged Android Apps: Literature Review and Benchmark. *IEEE Transactions on Software Engineering* 47, 4 (2021), 676–693. doi:10.1109/TSE.2019.2901679

[43] Li Li, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. An Investigation into the Use of Common Libraries in Android Apps. In *IEEE International Conference on Software Analysis, Evolution, and Reengineering*. IEEE, Piscataway, NJ, USA, 403–414. doi:10.1109/SANER.2016.52

[44] Li Li, Tegawendé F. Bissyandé, Mike Papadakis, Siegfried Rasthofer, Alexandre Bartel, Damien Octeau, Jacques Klein, and Yves Le Traon. 2017. Static Analysis of Android Apps: A Systematic Literature Review. *Information and Software Technology* 88 (2017), 67–95. doi:10.1016/j.infsof.2017.04.001

[45] Li Li, Daoyuan Li, Tegawende F. Bissyande, Jacques Klein, Yves Le Traon, David Lo, and Lorenzo Cavallaro. 2017. Understanding Android App Piggybacking: A Systematic Study of Malicious Code Grafting. *IEEE Transactions on Information Forensics and Security* 12, 6 (2017), 1269–1284. doi:10.1109/TIFS.2017.2656460

[46] Menghao Li, Wei Wang, Pei Wang, Shuai Wang, Dinghao Wu, Jian Liu, Rui Xue, and Wei Huo. 2017. LibD: Scalable and Precise Third-Party Library Detection in Android Markets. In *International Conference on Software Engineering*. IEEE, Piscataway, NJ, USA, 335–346. doi:10.1109/ICSE.2017.38

[47] Mario Linares-Vasquez, Andrew Holtzhauer, and Denys Poshyvanyk. 2016. On Automatically Detecting Similar Android Apps. In *IEEE International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 1–10. doi:10.1109/ICPC.2016.7503721

[48] Ziang Ma, Haoyu Wang, Yao Guo, and Xiangqun Chen. 2016. LibRadar: Fast and Accurate Detection of Third-Party Libraries in Android Apps. In *International Conference on Software Engineering Companion*. IEEE, Piscataway, NJ, USA, 653–656. doi:10.1145/2889160.2889178

[49] Enrico Mariconti, Lucky Onwuzurike, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2017. MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models. In *Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 15 pages.

[50] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2353–2367. doi:10.1145/3548606.3560596

[51] Kirill Nikitin, Eleftherios Kokoris-Kogias, Philipp Jovanovic, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Justin Cappos, and Bryan Ford. 2017. CHAINIAC: Proactive Software-Update Transparency via Collectively Signed Skipchains and Verified Builds. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 17 pages.

[52] Liang Niu, Shujaat Mirza, Zayd Maradni, and Christina Pöpper. 2023. CodexLeaks: Privacy Leaks from Code Generation Language Models in GitHub Copilot. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 2133–2150.

[53] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, Cham, Switzerland, 23–43. doi:10.1007/978-3-030-52683-2_2

[54] Marc Ohm and Christopher Stuke. 2023. SoK: Practical Detection of Software Supply Chain Attacks. In *International Conference on Availability, Reliability and Security*. Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/3600160.3600162

[55] Chinenye Okafor, Taylor R. Schorlemmer, Santiago Torres-Arias, and James C. Davis. 2022. SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. In *ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. Association for Computing Machinery, New York, NY, USA, 15–24. doi:10.1145/3560835.3564556

[56] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *IEEE Symposium on Security and Privacy*. IEEE, Piscataway, NJ, USA, 754–768. doi:10.1109/SP46214.2022.9833571

[57] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157

[58] Rahul Potharaju, Andrew Newell, Cristina Nita-Rotaru, and Xiangyu Zhang. 2012. Plagiarizing Smartphone Applications: Attack Strategies and Defense Techniques. In *International Symposium on Engineering Secure Software and Systems*. Springer, Cham, Switzerland, 106–120. doi:10.1007/978-3-642-28166-2_11

[59] Dezhi Ran, Yuan Cao, Mengzhou Wu, Simin Chen, Yuzhe Guo, Jun Ren, Zihe Song, Hao Yu, Jialei Wei, Linyi Li, Wei Yang, Baishakhi Ray, and Tao Xie. 2025. AppForge: From Assistant to Independent Developer—Are GPTs Ready for Software Development? arXiv:2510.07740.

[60] Vaibhav Rastogi, Yan Chen, and Xuxian Jiang. 2013. DroidChameleon: Evaluating Android Anti-Malware against Transformation Attacks. In *ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 329–334. doi:10.1145/2484313.2484355

[61] Beatriz M. Reichert and Rafael R. Obelheiro. 2024. Software Supply Chain Security: A Systematic Literature Review. *International Journal of Computers and Applications* 46, 10 (2024), 853–867. doi:10.1080/1206212X.2024.2390978

[62] Chuangang Ren, Kai Chen, and Peng Liu. 2014. DroidMarking: Resilient Software Watermarking for Impeding Android Application Repackaging. In *IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, 635–646. doi:10.1145/2642937.2642977

[63] Israel J. Mojica Ruiz, Meiyappan Nagappan, Bram Adams, and Ahmed E. Hassan. 2012. Understanding Reuse in the Android Market. In *IEEE International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 113–122. doi:10.1109/ICPC.2012.6240477

[64] Pasquale Salza, Fabio Palomba, Dario Di Nucci, Cosmo D'Uva, Andrea De Lucia, and Filomena Ferrucci. 2018. Do Developers Update Third-Party Libraries in Mobile Apps?. In *IEEE International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 255–265. doi:10.1145/3196321.3196341

[65] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. 2010. Survivable Key Compromise in Software Update Systems. In *ACM Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 61–72. doi:10.1145/1866307.1866315

[66] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 2205–2222.

[67] Yuru Shao, Xiapu Luo, Chenxiong Qian, Pengfei Zhu, and Lei Zhang. 2014. Towards a Scalable Resource-Driven Approach for Detecting Repackaged Android Applications. In *Annual Computer Security Applications Conference*. Association for Computing Machinery, New York, NY, USA, 56–65. doi:10.1145/2664243.2664275

[68] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. 2025. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 3687–3706.

[69] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Massimiliano Di Penta, Daniel M. German, and Denys Poshyvanyk. 2024. BOMs Away! Inside the Minds of Stakeholders: A Comprehensive Study of Bills of Materials for Software Systems. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3597503.3623347

[70] Sufatrio, Darell J. J. Tan, Tong-Wei Chua, and Vrizlynn L. L. Thing. 2015. Securing Android: A Survey, Taxonomy, and Challenges. *Comput. Surveys* 47, 4 (2015), 58:1–58:45. doi:10.1145/2733306

[71] Mingshen Sun, Mengmeng Li, and John C. S. Lui. 2015. DroidEagle: Seamless Detection of Visually Similar Android Apps. In *ACM Conference on Security and Privacy in Wireless and Mobile Networks*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/2766498.2766508

[72] Kimberly Tam, Ali Feizollah, Nor Badrul Anuar, Rosli Salleh, and Lorenzo Cavallaro. 2017. The Evolution of Android Malware and Android Analysis Techniques. *Comput. Surveys* 49, 4 (2017), 76:1–76:41. doi:10.1145/3017427

[73] Wei Tang, Peng Luo, Jingwei Fu, and Dinghao Zhang. 2020. LibDX: A Cross-Platform and Accurate System to Detect Third-Party Libraries in Binary Code. In *IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Piscataway, NJ, USA, 104–115. doi:10.1109/SANER48275.2020.9054845

[74] Ke Tian, Danfeng Yao, Barbara G. Ryder, Gang Tan, and Guojun Peng. 2020. Detection of Repackaged Android Malware with Code-Heterogeneity Features. *IEEE Transactions on Dependable and Secure Computing* 17, 1 (2020), 64–77. doi:10.1109/TDSC.2017.2745575

[75] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 1393–1410.

[76] Haoyu Wang, Yao Guo, Ziang Ma, and Xiangqun Chen. 2015. WuKong: A Scalable and Accurate Two-Phase Approach to Android App Clone Detection. In *ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, 71–82. doi:10.1145/2771783.2771795

[77] Yan Wang, Haowei Wu, Hailong Zhang, and Atanas Rountev. 2018. ORLIS: Obfuscation-Resilient Library Detection for Android. In *IEEE/ACM International Conference on Mobile Software Engineering and Systems*. Association for Computing Machinery, New York, NY, USA, 13–23. doi:10.1145/3197231.3197248

[78] Takuya Watanabe, Mitsuaki Akiyama, Fumihiro Kanei, Eitaro Shioji, Yohei Takata, Bo Sun, Yuta Ishii, Toshiki Shibahara, Takeshi Yagi, and Tatsuya Mori. 2017. Understanding the Origins of Mobile App Vulnerabilities: A Large-Scale Measurement Study of Free and Paid Apps. In *IEEE/ACM International Conference on Mining Software Repositories*. IEEE, Piscataway, NJ, USA, 14–24. doi:10.1109/MSR.2017.23

[79] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, Michel Cukier, Christian Kästner, Alexandros Kapravelos, Dominik Wermke, and William Enck. 2025. Research Directions in Software Supply Chain Security. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 146:1–146:38. doi:10.1145/3714464

[80] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/2601248.2601268

[81] Yiming Wu, Zhuowu Liu, Yanjie Lin, Binbin Zhao, Chunyi Zhou, Tiejun Wu, and Zhen Hong. 2026. Android App Repackaging Detection: A Comprehensive Survey. *Cyber Security and Applications* 4 (2026), 100124. doi:10.1016/j.csa.2026.100124

[82] Jian Xu and Qianting Yuan. 2022. LibRoad: Rapid, Online, and Accurate Detection of TPLs on Android. *IEEE Transactions on Mobile Computing* 21, 1 (2022), 167–180. doi:10.1109/TMC.2020.3003336

[83] Meng Xu, Chengyu Song, Yang Ji, Ming-Wei Shih, Kangjie Lu, Cong Zheng, Ruian Duan, Yeongjin Jang, Byoungyoung Lee, Chenxiong Qian, Sangho Lee, Taesoo Kim, and Wenke Lee. 2016. Toward Engineering a Secure Android Ecosystem: A Survey of Existing Techniques. *Comput. Surveys* 49, 2 (2016), 38:1–38:47. doi:10.1145/2963145

[84] Shengtao Yue, Weizan Feng, Jun Ma, Yanyan Jiang, Xianping Tao, Chang Xu, and Jian Lu. 2017. RepDroid: An Automated Tool for Android Application Repackaging Detection. In *IEEE/ACM International Conference on Program Comprehension*. IEEE, Piscataway, NJ, USA, 132–142. doi:10.1109/ICPC.2017.16

[85] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What Are Weak Links in the npm Supply Chain?. In *International Conference on Software Engineering: Software Engineering in Practice*. IEEE, Piscataway, NJ, USA, 331–340. doi:10.1145/3510457.3513044

[86] Jia Zeng, Dan Han, Yaling Zhu, Yangzhong Wang, and Fangchen Weng. 2024. A Survey of Third-Party Library Security Research in Application Software. arXiv:2404.17955.

[87] Xian Zhan, Lingling Fan, Sen Chen, Feng Wu, Tianming Liu, Xiapu Luo, and Yang Liu. 2021. ATVHunter: Reliable Version Detection of Third-Party Libraries for Vulnerability Identification in Android Applications. In *International Conference on Software Engineering*. IEEE, Piscataway, NJ, USA, 1695–1707. doi:10.1109/ICSE43902.2021.00150

[88] Xian Zhan, Lingling Fan, Tianming Liu, Sen Chen, Li Li, Haoyu Wang, Yifei Xu, Xiapu Luo, and Yang Liu. 2020. Automated Third-Party Library Detection for Android Applications: Are We There Yet?. In *IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 919–930. doi:10.1145/3324884.3416582

[89] Xian Zhan, Tianming Liu, Lingling Fan, Li Li, Sen Chen, Xiapu Luo, and Yang Liu. 2022. Research on Third-Party Libraries in Android Apps: A Taxonomy and Systematic Literature Review. *IEEE Transactions on Software Engineering* 48, 10 (2022), 4181–4213. doi:10.1109/TSE.2021.3114381

[90] Fangfang Zhang, Heqing Huang, Sencun Zhu, Dinghao Wu, and Peng Liu. 2014. ViewDroid: Towards Obfuscation-Resilient Mobile Application Repackaging Detection. In *ACM Conference on Security and Privacy in Wireless and Mobile Networks*. Association for Computing Machinery, New York, NY, USA, 25–36. doi:10.1145/2627393.2627395

[91] Jiexin Zhang, Alastair R. Beresford, and Stephan A. Kollmann. 2019. LibID: Reliable Identification of Obfuscated Third-Party Android Libraries. In *ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, 55–65. doi:10.1145/3293882.3330563

[92] Yiming Zhang, Jian Dai, Xiaohan Zhang, Shuang Huang, Zhen Yang, Min Yang, and Hao Chen. 2018. Detecting Third-Party Libraries in Android Applications with High Precision and Recall. In *IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Piscataway, NJ, USA, 141–152. doi:10.1109/SANER.2018.8330204

[93] Zhengzi Zhang, Wenrui Diao, Chao Hu, Shengjian Guo, Chaoshun Zuo, and Li Li. 2020. An Empirical Study of Potentially Malicious Third-Party Libraries in Android Apps. In *ACM Conference on Security and Privacy in Wireless and Mobile Networks*. Association for Computing Machinery, New York, NY, USA, 144–154. doi:10.1145/3395351.3399346

[94] Yury Zhauniarovich, Olga Gadyatskaya, Bruno Crispo, Francesco La Spina, and Ermanno Moser. 2014. FSquaDRA: Fast Detection of Repackaged Applications. In *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, Cham, Switzerland, 130–145. doi:10.1007/978-3-662-43936-4_9

[95] Wu Zhou, Xinwen Zhang, and Xuxian Jiang. 2013. AppInk: Watermarking Android Apps for Repackaging Deterrence. In *ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/2484313.2484315

[96] Wu Zhou, Yajin Zhou, Xuxian Jiang, and Peng Ning. 2012. Detecting Repackaged Smartphone Applications in Third-Party Android Marketplaces. In *ACM Conference on Data and Application Security and Privacy*. Association for Computing Machinery, New York, NY, USA, 317–326. doi:10.1145/2133601.2133640

[97] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *USENIX Security Symposium*. USENIX, Berkeley, CA, 995–1010.